Empirical Software Engineering (2023) 28:143
https://doi.org/10.1007/s10664-023-10382-z

Studying the characteristics of AIOps projects on GitHub

Roozbeh Aghili[1] · **Heng Li**[1] · **Foutse Khomh**[1]

Accepted: 14 August 2023 / Published online: 18 October 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract
Artificial Intelligence for IT Operations (AIOps) leverages AI approaches to handle the massive amount of data generated during the operations of software systems. Prior works have proposed various AIOps solutions to support different tasks in system operations and maintenance, such as anomaly detection. In this study, we conduct an in-depth analysis of open-source AIOps projects to understand the characteristics of AIOps in practice. We first carefully identify a set of AIOps projects from GitHub and analyze their repository metrics (e.g., the used programming languages). Then, we qualitatively examine the projects to understand their input data, analysis techniques, and goals. Finally, we assess the quality of these projects using different quality metrics, such as the number of bugs. To provide context, we also sample two sets of baseline projects from GitHub: a random sample of machine learning projects and a random sample of general-purposed projects. By comparing different metrics between our identified AIOps projects and these baselines, we derive meaningful insights. Our results reveal a recent and growing interest in AIOps solutions. However, the quality metrics indicate that AIOps projects suffer from more issues than our baseline projects. We also pinpoint the most common issues in AIOps approaches and discuss potential solutions to address these challenges. Our findings offer valuable guidance to researchers and practitioners, enabling them to comprehend the current state of AIOps practices and shed light on different ways of improving AIOps' weaker aspects. To the best of our knowledge, this work marks the first attempt to characterize open-source AIOps projects.

Keywords AIOps · Data mining · Repository mining · Qualitative analysis · Temporal trends · Source code analysis

Communicated by: Markus Borg

✉ Roozbeh Aghili
roozbeh.aghili@polymtl.ca

Heng Li
heng.li@polymtl.ca

Foutse Khomh
foutse.khomh@polymtl.ca

[1] Department of Computer Engineering and Software Engineering, Polytechnique Montreal, Montreal, QC, Canada

1 Introduction

With the prevalence of generated data in large-scale systems, monitoring this data and transforming it into practical insights is becoming a complex challenge. Artificial Intelligence for IT Operations (AIOps) has been introduced to cope with this challenge. It combines big data, machine learning (ML) approaches, and other advanced analysis techniques (e.g., statistical analysis) to analyze system performance patterns to be able to improve service quality and reduce operational costs (Dang et al. 2019; Prasad and Rich 2018). By utilizing AIOps techniques, organizations are now able to collect and combine different sources of system data and use them to perform various tasks (e.g., anomaly detection or failure prediction) in their DevOps or operations environment (Dang et al. 2019; Prasad and Rich 2018).

A considerable amount of research has been performed on the topic of AIOps. Prior work has proposed AIOps solutions for various maintenance and operations tasks, such as predicting node failures (Lin et al. 2018; Li et al. 2020b), predicting task or job failures (Gao et al. 2020; El-Sayed et al. 2017; Rosá et al. 2015), anomaly detection (He et al. 2018b; Lim et al. 2014), and self healing (Ding et al. 2012, 2014). However, no work has systematically studied AIOps practices in real-world projects (e.g., public GitHub projects).

Studying AIOps practices in real-world projects is important and has several benefits, including (1) helping researchers and practitioners understand the current status of AIOps solutions and the characteristics of AIOps projects; (2) providing guidance for researchers and practitioners to adopt best–performing AIOps solutions for their application scenarios; and (3) identifying problems in AIOps practices and shedding lights on future research opportunities.

Therefore, this work identifies and studies a set of AIOps projects publicly available on GitHub. We also compare the selected AIOps projects with two baselines: traditional ML projects and General–purpose projects. Our goal is to understand the characteristics of these AIOps projects in the context of the baseline projects. Specifically, we first investigate the overall characteristics of these AIOps projects in terms of their GitHub metrics (RQ1), then we dig deeper into the individual projects to understand "what" goals these projects aim to achieve (RQ2), and finally, we examine how well these goals are achieved in terms of the code quality of these projects (RQ3). Our research questions are as follows.

RQ1 ***What are the characteristics of AIOps projects in terms of their GitHub metrics?*** We analyze the GitHub metrics of AIOps and baseline projects to understand the current status of AIOps projects and also compare them with baselines in terms of their GitHub metrics, such as the programming languages and the number of stars. We observe that AIOps solutions are being developed with a faster growth rate compared to the baselines. AIOps projects also have a higher distribution of popularity metrics (e.g., number of stars and forks), and also more pull requests and issues compared to baselines.

RQ2 ***What are the characteristics of AIOps projects in terms of their input data, analysis techniques, and goals?*** In order to understand the characteristics of AIOps projects (i.e., their input data, analysis techniques, and goals), we manually investigate each project. We find that monitoring data (e.g., logs and performance metrics) is the most used input data, classical machine learning techniques are the most adopted analysis techniques, and anomaly detection is the primary goal of many AIOps projects.

RQ3 ***What is the code quality of AIOps projects?*** We further analyze the source code of AIOps and baseline projects to identify any interesting patterns related to their quality. We find that AIOps projects have a higher issue rate, specifically in terms of bugs, code smells, and technical debt, than the baselines.

We share our replication package on GitHub [1] so that future work can reproduce or extend our study. Our work makes several important contributions:

1. As the first study on AIOps practices in real-world projects, our work helps practitioners and researchers understand the status of AIOps from a practical point of view.
2. Our qualitative analysis of the input data, analysis techniques, and goals of the AIOps projects can help practitioners and researchers consider and adopt AIOps solutions that fit into their specific application scenarios.
3. Our work identifies problems in AIOps practices (e.g., code quality) and sheds light on future research opportunities in AIOps.

The rest of the paper is organized as follows. Section 2 describes the experiment setup of our study, including the collection and preparation of the AIOps project data used for answering our research questions. Section 3 presents our approach and results for answering the research questions. Section 4 provides further discussions of our results. Section 5 discusses the threats to the validity of our findings. Section 6 summarizes prior research related to our work, and finally, Sect. 7 concludes our paper.

2 Experiment Setup

This section describes our approach for collecting the AIOps and baseline projects. We first present the overview of our study, then describe the steps for collecting and verifying AIOps and baseline projects, respectively.

2.1 Overview of Our Study

Figure 1 presents an overview of our approach to study the characteristics of AIOps projects. We use GitHub as the main source to extract the needed data for our analysis. As of January 2023, GitHub has over 100 million registered developers and over 372 million repositories (GitHub 2023). GitHub is also considered the largest hosting service for open-source software systems (Li et al. 2020a). Hence, projects found on GitHub are likely to reflect the diversity of existing AIOps projects. Many existing studies also extract the needed information for their analysis from GitHub (e.g., Dakhel et al. 2023; Foalem et al. 2023; Majidi et al. 2022; Openja et al. 2022). We start by searching the projects with the keyword "AIOps". Then, through manual verification (e.g., removing non-AIOps projects), keyword expansion (i.e., through pattern mining), second-round search and manual verification, and threshold-based filtering (e.g., by the number of stars), we collect a total of 119 AIOps projects that are used to answer our research questions. In order to better understand the characteristics of AIOps projects in a bigger context, we also compare our identified AIOps projects with two baselines:

1. randomly sampled machine learning (ML) projects;
2. randomly sampled general projects

We choose the ML baseline because most of the AIOps projects leverage ML techniques. We choose the General baseline to compare our AIOps projects with general software applications on GitHub. Finally, we perform qualitative and quantitative analyses on the collected data to answer our research questions. Below, we describe the details of our data collection. The detailed approaches for answering our research questions are presented in Sect. 3.

[1] https://github.com/roozbehaghili/studying_aiops_github

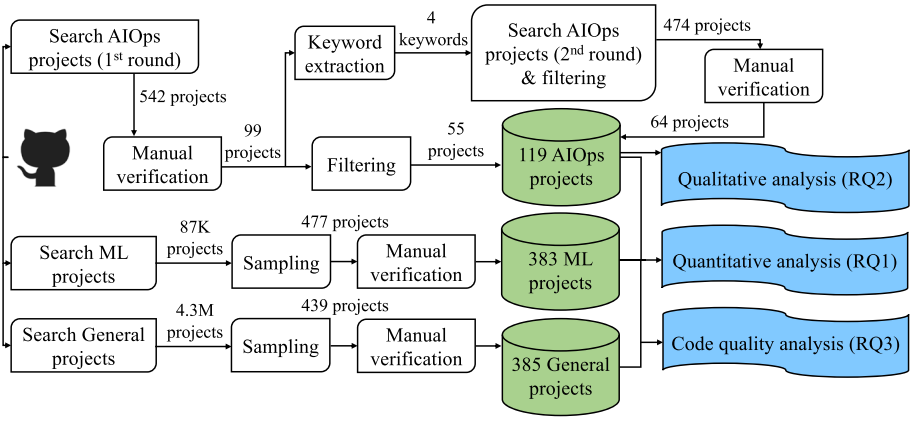


Fig. 1 An overview of our study

2.2 Collecting AIOps Projects

Through two rounds of searching AIOps-related keywords on GitHub, we collect a total of 1016 candidate projects. Through filtering and manual verification, we end up with 119 of them as our final set of AIOps projects. The methodology we use to select these projects follows the systematic approach recommended by Basili et al. (1986) and is described in the following sections.

2.2.1 Search AIOps Projects (First Round)

In the initial step of identifying suitable projects, we utilize the GitHub interface to search for repositories specifically labeled as AIOps projects. To do this, we employ the keyword "AIOps" and search across four key sections of each repository: the repository name, the "about" section, the "topics" section, and the contents of the "readme" file. After searching for the keyword "AIOps", we find a total of 542 repositories that match our criteria. These 542 repositories represent all the available projects on GitHub that have been labeled as AIOps projects.

2.2.2 Manual Verification (First Round)

Based on existing definitions of AIOps (Dang et al. 2019; Prasad and Rich 2018), we consider AIOps projects as: *any project that uses IT Operations-related data, utilizes advanced analysis technologies such as machine-learning or statistical analysis to reach valuable insights or enhance the system's quality by actions such as monitoring and automation.*

Therefore, not all the 542 discovered repositories are good AIOps candidates and suitable for our study. We hence select our subject projects based on three criteria:

1. The project should be about AIOps, not similar topics such as Machine Learning Operations (MLOps)[2] or Development and Operations (DevOps).[3] Therefore, we delete the projects that are mainly about other topics.

[2] A set of practices to maintain and deploy machine learning models.

[3] A set of practices that aim to shorten the system development life cycle while preserving high quality.

2. The projects should contain sufficient code. Therefore, we delete the repositories that do not have any code or only contain a few lines of code. We also delete projects that are a collection of papers, slides, or other repositories. However, we accept the projects that have created a dataset so that other developers and researchers can use it in their work.
3. The projects should not be toy projects: we delete the projects that are homework assignments or university projects.

To select the desirable projects based on the explained criteria, the first two authors of the paper (i.e., coders) independently perform a coding process, adding a YES (AIOps projects) or NO (non-AIOps projects) tag to each project. We perform the coding process as below.

Step 1: Coding. Each coder studies and analyzes all the 542 repositories and independently decides if each project should be added to the final list of projects.
Step 2: Discussion. The coders share their responses and discuss their approaches for selecting a project. The discussion session's goal is to reach the same understanding of the inclusion criteria among the coders.
Step 3: Revision. Based on the discussion, each coder revises their responses from step 1.
Step 4: Resolving disagreements. In the last step, the coders discuss any conflicts that may remain and try to resolve them. If an agreement can not be reached, the third author would analyze the project, and a vote is performed.

After performing the manual verification process, we obtain a total of 99 candidate AIOps projects from the 542 projects derived from the search results, which corresponds to a selection rate of 18%.

2.2.3 Keyword Extraction (Pattern Mining)

Expanding our search to gain a more comprehensive view of real-world AIOps projects is necessary as the term "AIOps" was only introduced in 2018 (Prasad and Rich 2018). As a result, some projects might have existed prior to the introduction of this terminology, implementing AIOps solutions without explicitly using the exact term. To achieve a broader scope, we extend our search to include additional projects that may not explicitly label themselves as AIOps repositories but are, in fact, implementing AIOps solutions. To achieve this, we extract all the topics associated with each of the 99 AIOps projects resulting from the previous step. These topics can be found in the "topics" section of each GitHub repository.

We then use frequent pattern mining (Han et al. 2007), a method aimed at discovering associations and patterns within a given dataset. Specifically, we utilize the frequent pattern growth technique (Han et al. 2000, 2004) to identify the most common topics among GitHub repositories. To conduct this analysis, we set the support parameter to 2, indicating that a pattern should appear in at least two projects for consideration. Through this approach, we identify a total of 194 patterns among the topics present in the repositories. Next, we perform a discussion session involving all three authors to decide which patterns hold potential for identifying additional AIOps projects. From this discussion, we narrow down our selection to four pairs of two-item patterns: "anomaly detection" and "log analysis", "log analysis" and "machine learning", "anomaly detection" and "machine learning", as well as "machine learning" and "metrics". All the selected keyword pairs are among the most frequently used topics in the projects. We use these four sets of keywords to find more AIOps repositories.

2.2.4 Search AIOps Projects (Second Round)

Using the four pairs of keywords obtained in the previous step, we conduct the second round of search on the GitHub interface to identify more projects related to AIOps. We follow the same process as described in Sect. 2.2.1. After completing this second-round search and removing any duplicated projects that were already identified in the first-round search results, we find a total of 474 unique projects.

2.2.5 Filtering

Based on the knowledge that we have gained from our first-round manual verification, we apply a filtering phase to remove the toy projects. To achieve this, we employ two filtering criteria based on the number of stars and forks for each project. Specifically, we only consider projects that have both stars and forks greater than or equal to 1 (stars: >=1 & forks: >=1). The purpose of removing toy projects is to have relatively mature projects and not soil our results with small repositories (Munaiah et al. 2017). Given the limited number of AIOps projects on GitHub, adding stricter filtering criteria would result in much fewer projects. Therefore, we choose a low-bar filtering approach to reduce the manual effort of analyzing all the projects with stars or forks of 0.

We utilize the filtering process for both the projects from the first and second rounds of searches. In the first round, we manually verify all the projects before applying the filtering step. This deliberate approach allows us to gain a comprehensive understanding of the status of the AIOps projects. Following the filtering process, the initially verified 99 projects are reduced to 55. For the projects from the second round, we opt to apply the filtering before the manual verification process. This decision saves unnecessary manual effort by excluding projects that do not meet the filtering criteria from the beginning.

2.2.6 Manual Verification (Second Round)

As detailed in Sect. 2.2.2, not all of the repositories obtained from our initial search are suitable for our study. To carefully select the projects that align with our research objectives, we carry out a manual verification process, repeating the steps outlined in Sect. 2.2.2. During this verification, the coders examine all 474 projects that resulted from the expanded keyword search to determine their suitability as real AIOps projects. After steps of separate coding, discussion, revision, and resolving disagreements, we identify 64 projects from the expanded keyword search. This corresponds to a selection rate of 14%, Finally, we combine the 55 projects identified using the "AIOps" keyword with the 64 projects discovered through the expanded keywords. The final set of 119 AIOps projects is used to answer our research questions. The approaches for answering our research questions are detailed in Sect. 3.

2.2.7 Measuring the Reliability of Our Manual Verification

Reliability is vital to ensure the validity of the coding results (Artstein and Poesio 2008). The coding results are reliable if there exists a certain level of agreement between the coders, known as inter-coder agreement. In this study, we use Cohen's kappa (Cohen 1960) to measure the reliability of the agreements between two coders. Cohen's kappa is one of the most common approaches to measuring the inter-coder agreement (Artstein and Poesio 2008). Table 1

Table 1 Interpretation of Cohen's kappa

Value of Cohen's k	Level of Agreement
0-.20	None
.21-.39	Minimal
.40-.59	Weak
.60-.79	Moderate
.80-.90	Strong
.90-1	Almost Perfect

indicates the relation between the value of Cohen's kappa and the level of agreement (McHugh 2012).

Our manual verification for choosing the proper AIOps projects achieves a Cohen's kappa of 0.68 before the discussion session. After the discussion session between the coders, the kappa score increased to 0.84. As shown in Table 1, kappa ≥ 0.80 indicates a strong agreement.

2.3 Collecting Baseline Projects

To understand how AIOps projects differ from traditional software projects, we create two baselines and compare the AIOps projects with them. We select Machine Learning (ML) projects as our first baseline and General projects as our second baseline.

2.3.1 Machine Learning Projects

For our initial baseline, we choose ML projects. We select ML for the first baseline because AIOps can be considered as an application domain of machine learning. In similar studies, researchers typically search for specific frameworks to gather ML projects on GitHub. For example, Zhang et al. (2018) select their project set by searching the keyword "TensorFlow", while Islam et al. (2019) gather their projects using various keywords such as "TensorFlow" and "Keras". However, as our focus is not limited to any specific framework, we use more general keywords. To gather our set of ML baseline projects, we utilize two keywords: "machine learning" and "deep learning." This approach allows us to cast a broader net and capture a comprehensive range of relevant ML projects for comparison and analysis.

To compare the AIOps projects with the baselines in a similar context, we apply a similar filtering process to the baseline projects. Like the AIOps projects, we only extract ML projects that have at least one star and one fork. Additionally, we take into account that ML projects generally have a longer history than AIOps projects. To address this difference, we apply the same date range for the creation of the ML baseline projects as observed in the AIOps set. Specifically, the earliest and latest creation dates in the AIOps projects are 2012/12/25 and 2022/10/27, respectively. Thus, we apply the same date range to filter the ML baseline projects. Following the search and removal of duplicate projects in two queries, we obtain a total of 87,276 unique repositories for the ML baseline. Due to time and computational resource limitations, extracting and analyzing all these projects becomes impractical. Hence, following prior work (e.g., Chen et al. 2020a and Zhang et al. 2019), a sample of 383 projects is needed to represent the pool of 87,275 ML repositories with a confidence level of 95% and a confidence interval of 5%.

2.3.2 General Projects

As our second baseline, we choose general projects from GitHub, meaning that we do not focus on any particular topic in our search. We also do not limit the General baseline to a specific programming language (e.g., Python) since AIOps projects are not limited to a single language as well. In this way, the General baseline captures the general characteristics of all GitHub projects. We then apply the same filtering phase as we did for the ML baseline, but without indicating any specific topic. After completing the filtering, we obtain 4,358,342 public and available repositories for the General baseline. Similar to the ML baseline, a sample of 385 projects is required to represent the pool of 4,358,342 General repositories with a confidence level of 95% and confidence interval of 5%.

2.3.3 Manual Verification

In order to maintain consistency with our methodology for selecting AIOps projects, we apply manual verification to our two baseline sets as well. The selection of baseline projects is based on two key criteria:

1. The projects should contain sufficient code and be mature. Therefore, we delete the repositories that do not have code and are a collection of papers, slides, or other repositories.
2. The projects should not be toy projects: we delete the projects that are homework assignments or university projects.

To construct our baseline sets, we randomly extract 500 projects from both ML and General pool of repositories. We then divide each set of baseline projects into two parts of 20% and 80% portions. Similar to Sect. 2.2.2, the two coders independently label 20% of each baseline to ensure a reliable assessment. We then measure the reliability of our coding using Cohen's kappa (Cohen 1960). The results of our manual verification for choosing the ML and General baselines indicate a strong agreement between the coders, with Cohen's kappa scores of 0.81 and 0.91, respectively. As the measurement shows a strong agreement between coders, the rest of baseline projects (the 80% portions) are labeled by the first coder.

We continue our manual labeling process until we obtain the statistical representative set for each baseline. In the case of ML projects, we thoroughly analyze 477 projects, ultimately selecting 383 of them to create our final set of ML baseline. Similarly, for the General projects, we thoroughly analyze 439 projects, resulting in the selection of 385 projects as our final set of General baseline.

As shown in Fig. 1, we compare our AIOps projects with the baseline projects in RQ1 where we study the repositories statistics of the projects, as well as in RQ3 where we study the code quality of the projects. In RQ2, we perform a qualitative study for the AIOps projects only, since the research question is specifically about AIOps projects (i.e., the input data, analysis techniques, and goals of AIOps solutions).

3 Research Questions and Results

This section presents the details of our research questions (RQs) and the results. We organize each RQ by its motivation, approach, and results.

3.1 RQ1. What are the Characteristics of AIOps Projects in Terms of their GitHub Metrics?

3.1.1 Motivation

Prior studies proposed AIOps solutions that leverage AI technologies to support various software operation efforts (Di Stefano et al. 2021; Li et al. 2022a, 2020b). However, no work has investigated real-world AIOps projects and their characteristics. Thus, this RQ bridges the gap to study the characteristics of AIOps projects and compare them with the baseline projects in terms of their GitHub metrics. With this comparison, we can understand the similar and different patterns between the characteristics of AIOps projects and the baselines. Our findings can help AIOps researchers and practitioners understand the state of AIOps in practice.

3.1.2 Approach

In this RQ, we analyze the repository characteristics of AIOps projects and compare them with the baseline projects. We use We write a script to extract the metrics of both AIOps and baseline projects. We use GitHub REST API (GitHub 2022) to retrieve the repository characteristics of these projects. In particular, we analyze the characteristics of AIOps projects and the baseline projects from three perspectives: growth of repositories, programming languages, and repository metrics.

Growth of repositories To understand the evolution of the population of AIOps projects, we analyze the distribution of the AIOps projects based on their creation time. We also compare the creation time distribution with that of the baseline projects.

Programming languages Developers may use different programming languages for AIOps projects. Understanding the distribution of the programming languages can provide insights for future work to support AIOps project development. Each project may use multiple programming languages. In this work, we extract and present the primary language of each project.

Repository metrics We study the repository metrics of the AIOps projects, including the number of stars, forks, commits, contributors, releases, pull requests, issues, size, and the status of being archived. Similarly, for the issues, we sum the open and closed issues.

Statistical tests We further perform statistical tests to evaluate the statistically significant difference between metrics for AIOps and baseline projects. We first conduct the *Shapiro-Wilk* test (Shapiro and Wilk 1965) to test the normality of our metrics. Using the widely accepted 0.05 significance threshold, the Shapiro-Wilk test shows that the GitHub metrics of AIOps and baseline projects do not follow a normal distribution. Since our data is not normally distributed, we select nonparametric tests. We use *Mann–Whitney U* test (Mann and Whitney 1947) to compare our samples. We also use *Cliff's delta* test (Cliff 1993) to test the effect size between our samples. Regarding Mann-Whiteny U test, we use the significance threshold of 0.05. Regarding Cliff's delta test, we use the scale presented by Romano et al. (2006), that effect of $|d| = 0.147$ is small, $|d| = 0.33$ is medium, and $|d| = 0.474$ is large.

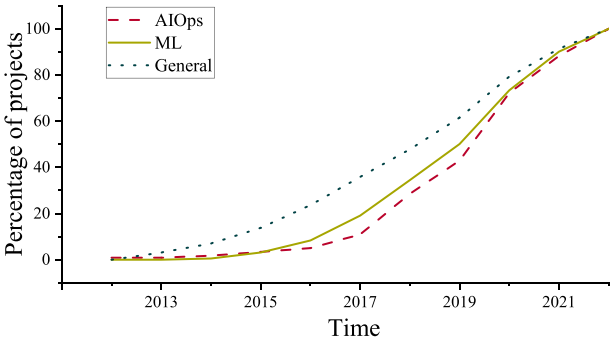


Fig. 2 The cumulative distribution of the creation time of AIOps and baseline projects

3.1.3 Results

Compared to the ML and General baselines, AIOps projects are relatively new and exhibit rapid growth in recent years Fig. 2 represents the percentage of projects created in and before each year for the AIOps and baseline projects. As mentioned in Sect. 2.3, the creation date of all the projects is between 2012/12/25 and 2022/10/27. As shown in Fig. 2, in the first few years (from 2012 to 2017), the number of AIOps projects is very small. In recent years (from 2017 to 2022), the AIOps projects experience a faster growth compared to the ML and General baselines. As can be seen, the ML projects also exhibit a faster growth than the General baseline projects.

Like in ML projects, Python is the dominant programming language in the AIOps projects; however, unlike in ML projects, Java is also a major programming language used in AIOps projects As shown in Table 2, Python is the most used language among all the groups; however, the usage of Python is much higher in AIOps and ML repositories than in the General baseline (71.4% and 81.7% in contrast to 21.6%). Another interesting finding is the relatively high usage of Java in AIOps projects (10.1%), while Java is not among the top-5 popular languages in the ML baseline.

On average, AIOps projects are more popular and active than the baselines Fig. 3 represents the box plots of GitHub metrics for the AIOps projects and the baselines. Table 3

Table 2 The top-5 languages of AIOps and baseline projects

AIOps		ML		General	
Language	Usage (%)	Language	Usage (%)	Language	Usage (%)
Python	71.4	Python	81.7	Python	21.6
Java	10.1	HTML	2.6	JavaScript	16.4
Go	3.4	R	1.8	Java	8.1
HTML	2.5	C++	1.8	TypeScript	4.7
JavaScript	1.7	JavaScript	1.8	PHP	4.4

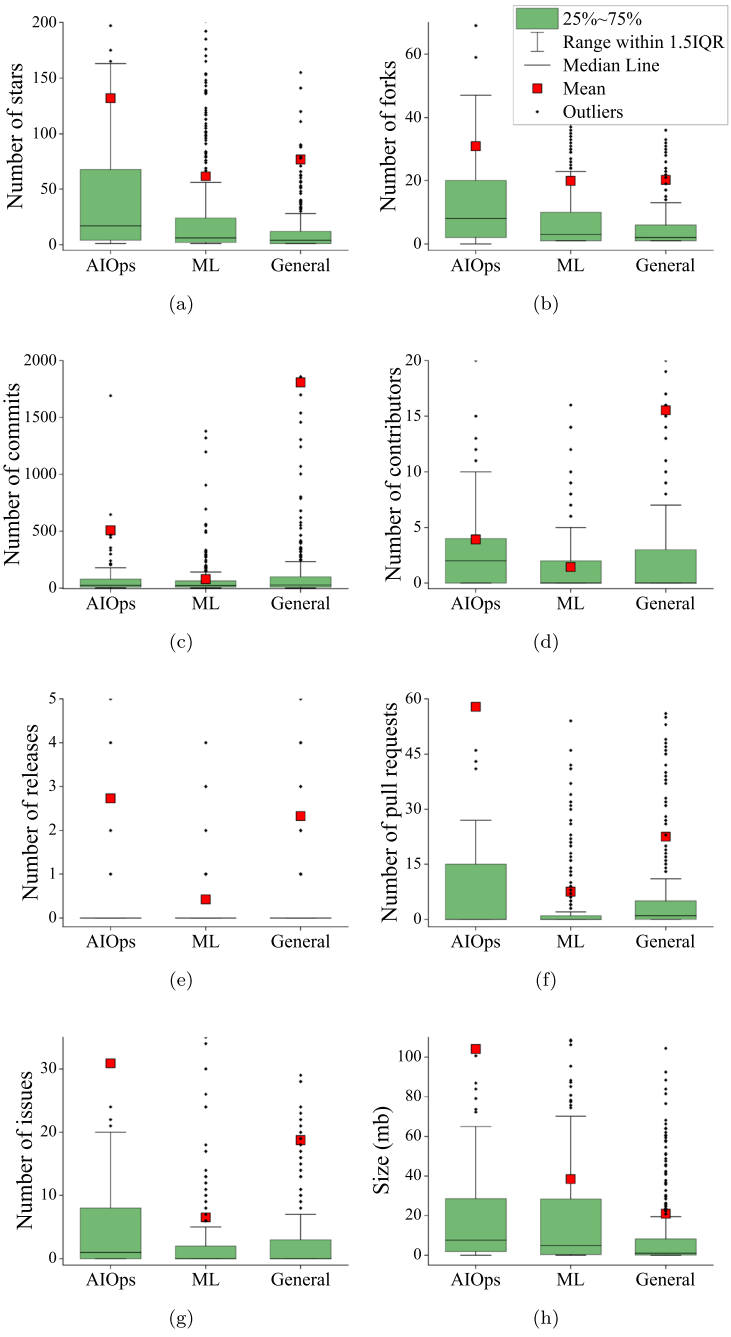


Fig. 3 Box plots of GitHub metrics for AIOps and baseline projects. The median number of 0 for the releases, pull requests, and issues indicate that more than half of the projects do not have any release, pull request, or issue

Table 3 Detailed results of *Mann–Whitney U* and *Cliff's delta* tests on projects' GitHub metrics

Metric	AIOps vs. ML		AIOps vs. General	
	p-value	effect size	p-value	effect size
Stars	0.00	**	0.00	***
Forks	0.00	**	0.00	***
Commits	0.34	-	0.92	-
Contributors	0.00	**	0.02	*
Releases	0.04	*	0.87	-
Pull requests	0.00	**	0.59	-
Issues	0.00	**	0.01	*
Size	0.02	*	0.00	***

Mann–Whitney U results are shown in *p-value* columns. If the sets have statistically different distributions, the *Cliff's delta* results are shown in *effect size* columns
*: negligible effect **: small effect ***: medium effect

shows the p-value and effect size of the GitHub metrics of AIOps projects compared to the baselines.

Considering the number of stars and forks, AIOps projects are more popular than the baselines, as both median and mean values in AIOps projects are higher than the baselines. In terms of the median value, AIOps projects have 4 times more stars and forks than the General baseline (median values of stars in AIOps and General projects are 17 and 4, and median values of forks in AIOps and General projects are 8 and 2). Regarding the statistical test results in Table 3, the p-value of stars and forks indicate that AIOps projects have statistically different distributions from ML and General baselines. For both of the metrics, the effect size is small compared to the ML baseline and is medium compared to the General baseline.

Regarding the number of commits, as shown in the statistical test results in Table 3, there is not a statistically significant difference between AIOps and baselines. Regarding the number of contributors, it seems that AIOps projects are more collaborative (median of 2 in AIOps and 0 in baselines). Statistical results also corroborate this finding, with the effect size of small compared to ML baseline and negligible (with a statistically significant difference) compared to General baseline. Regarding the number of releases, there is not a significant difference between AIOps and General baseline; however, AIOps projects have a statistically significant difference compared to ML baseline.

Comparing the number of pull requests and issues, as shown in Fig. 3f and 3g, we notice that AIOps projects experience more pull requests and issues. Statistical results also confirm this finding, as p-values of pull requests and issues are less than 0.05 (except pull requests for General baseline). This may be explained in three ways. First, the AIOps projects are more active and popular (i.e., more developers proactively develop them), leading to a more significant number of pull requests and issues. The second interpretation might be that AIOps projects are in the first stages of formation and not mature enough, having more defects and flaws and more developers trying to fix these problems. The third explanation is that AIOps projects are on average larger than the two baseline projects (as shown in Fig. 3h), which may lead to more pull requests and issues.

Comparing the size of the projects, AIOps projects tend to be larger than General baseline as the statistical tests indicate a medium effect size. AIOps projects also are larger than ML

projects; however, the difference is not as large as General baseline, as the statistical tests result in a negligible (with a statistically significant difference) effect size. Comparing the median values, the size of AIOps projects is 7 times larger than General baseline (7.6 MB for AIOps projects compared to 1.0 MB for General baseline).

We further analyze the percentage of projects that have been archived. The results indicate that only 1.7% of AIOps projects have been archived, while this amount for ML and General baseline is 1.8% and 4.2%, respectively.

Overall, taking into consideration all the mentioned metrics, AIOps projects seem to be more active and popular than both baselines.

Summary of RQ1

On average, AIOps projects are receiving more attention than the ML and General baselines. The primary language used in them is Python, followed by Java. They are growing faster than the baselines in recent years, demonstrating the growing needs and active practices in this area. The size of AIOps projects is larger than the baselines, and focusing on other GitHub metrics such as number of stars, forks, and releases, AIOps projects seem to be more popular and active than both baselines.

3.2 RQ2. What are the Characteristics of AIOps Projects in Terms of their Input Data, Analysis Techniques, and Goals?

3.2.1 Motivation

AIOps researchers and practitioners leverage different techniques to analyze different types of operational data and achieve different goals. However, it is unclear how real AIOps projects leverage data and technologies to achieve the goals. In this RQ, we qualitatively analyze our set of AIOps projects to understand the characteristics of these projects' input data, analysis techniques, and goals. Our results can help researchers and practitioners further understand the status of AIOps practices and the characteristics of AIOps projects. Our results can also provide insights for future work to provide support for different AIOps application scenarios.

3.2.2 Approach

We manually examine each AIOps project to understand its input data, analysis techniques, and goals. For each project, we manually investigate four sources of information; the "about" section, the "readme" file, the source code, and the additional documentations if available. Figure 4 illustrates the three key concepts of our manual analysis (input data, analysis techniques, and goals) and their relationship.

– **Input data:** The types of data (e.g., log data) that an AIOps project takes as inputs to achieve its objectives.

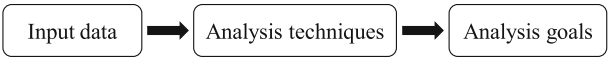


Fig. 4 The three key concepts of our manual analysis

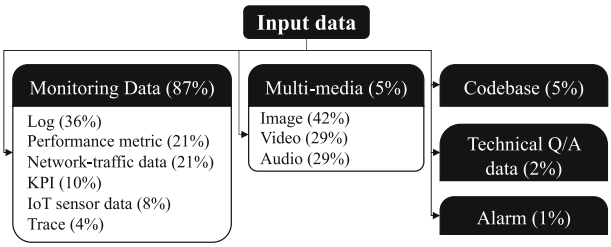


Fig. 5 The categorization of the input data used in AIOps projects. The high level categories are highlighted in dark

- **Analysis techniques:** The main analysis techniques (e.g., machine learning techniques) that an AIOps project adopts to analyze the input data and achieve its objectives.
- **Goals:** The objectives (e.g., anomaly detection) that an AIOps project aims to achieve through its input data and analysis techniques.

Manual coding process We use open coding approach (Khandkar 2009) to extract the information related to the three key concepts shown in Fig. 4. Open coding is widely used among software engineering researchers to conclude a high-level abstraction from lower-level data (Stol et al. 2016; Wohlin and Aurum 2015). To label the projects, the first two authors of the paper (i.e., coders) jointly perform a coding process, determining each project's input data, analysis techniques, and goals. We perform a five-step coding process as follows.

Step 1: Coding. Each coder analyzes the 97 AIOps projects and assign labels for each concept (input data, techniques, and goals) of each project. Multiple labels can be assigned to a concept of a project. This step takes a few days for each coder to complete.

Step 2: Discussion. The coders share their responses and discuss the created labels. The main goal of the discussion session is to obtain a common understanding of the labels for the input data, techniques, and goals. Based on the separate labels of the coders, we join related labels together and take apart some high-level labels into smaller ones. After this session, we finalize the labels for each concept (input data, techniques, and goals).

Step 3: Revision. Based on the results of the discussion session and the agreed-upon labels, each coder revises his responses from Step 1.

Step 4: Resolving disagreements. The coders compare their final results from step 3 and discuss any conflict that may remain. The coders try to resolve the conflicts, but if an agreement can not be reached, the third author analyzes the project, and the final decision is made.

Step 5: Final revision. In the final stage, we create a mind map from all the produced labels. We then discuss the labels and form an hierarchy, change some labels' names for clarity, and merge some small categories to be cohesive.

3.2.3 Results

Figs. 5, 6, and 7 present our categorization of the input data, analysis techniques, and goals of the AIOps projects, process, respectively. We further define our coding labels in Tables 4, 5, and 6. It is important to note that each project may have multiple input data, analysis techniques, or goals.

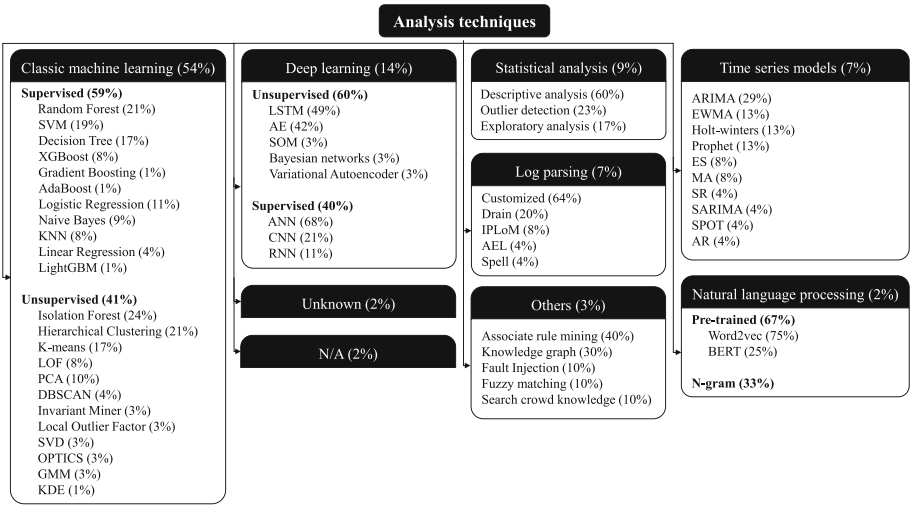


Fig. 6 The categorization of the analysis techniques (LSTM approaches can be used in a supervised or unsupervised manner (Chen et al. 2021). In our set of projects and primarily for anomaly detection, projects use LSTM in the unsupervised form) used in AIOps projects. The high level categories are highlighted in dark

Input data: Monitoring data (e.g., logs, performance metrics, and network-traffic data) is the dominant type of input data of the AIOps projects, with logs being the most commonly used input data type The categorization of the input data is illustrated in Fig. 5. We divide the input data of AIOps projects into five main categories: monitoring data, multimedia, codebase, technical question-answer (QA) data, and alarm. We define these categories

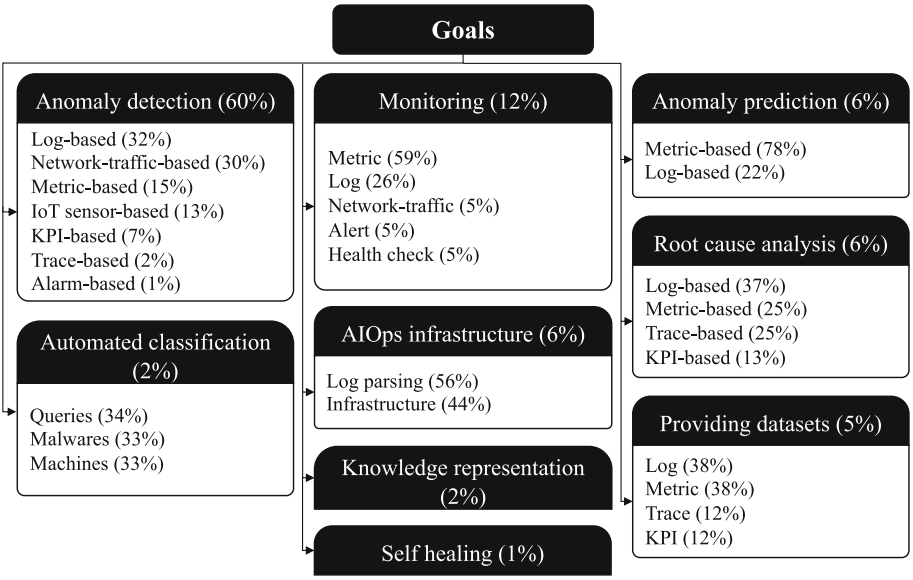


Fig. 7 The categorization of the goals of the AIOps projects. The high level categories are highlighted in dark

and provide an example for each in Table 4. Among them, monitoring data is the most popular, used in 87% of the projects. The monitoring data is divided into 6 sub-categories: log, performance metric, network-traffic data, Key Performance Indicator (KPI), Internet of Things (IoT) sensor data, and trace. Among them, log data is the most commonly used. These data types could be used for different purposes such as extensive monitoring, debugging, performance analysis, test analysis, and business analytics (Chen and Jiang 2021; Svoboda et al. 2015). In this paper, we use the umbrella term of "monitoring data" to name these data types.

We find two interesting input data types for the AIOps projects: network traffic and IoT sensor data. A considerable proportion of the projects (21%) use network traffic data as their input. Also, 8% of projects use IoT sensor data.

The second popular category of input data for AIOps approaches is multi-media (image, video, or audio) (5%), followed by codebase (5%), technical Q/A data (2%), and alarm (1%). We define the input of a project as "codebase" if it analyzes the source code or configuration files of other software. We define the input as "technical Q/A data" if the project analyzes data from Q/A websites like Stack Overflow.

Analysis techniques: Classical machine learning models are the most commonly used analysis techniques, followed far behind by deep learning, statistical analysis, and time series models We present our categorization of the applied analysis techniques of AIOps projects in detail in Fig. 6. We derive 9 high-level categories: classic machine learning (54%), deep learning (14%), statistical analysis (9%), time series models (7%), log parsing (7%), others (3%), natural language processing (2%), unknown (2%), and N/A (2%). Definitions and examples for each category can be found in Table 5.

Both the classic machine learning and deep learning categories are further divided into supervised and unsupervised learning algorithms.

Overall, considering supervised and unsupervised usage in classical machine learning and deep learning approaches, 37% of the projects in our study employ supervised learning, while 31% utilize unsupervised approaches. Hence, the difference between the adoption of supervised and unsupervised methods is not substantial. Therefore, we could not find a strong correlation with Dang et al. (2019), where they state that in *many* AIOps cases, only unsupervised machine learning models are practical. However, we find that some projects in our study explore a combination of supervised and unsupervised models, conducting comparisons to assess their performance for specific tasks. While unsupervised techniques outperforms the supervised approaches in scenarios where data labeling is limited or new patterns of data can emerge, there are also instances where supervised approaches achieve better results. For example, as mentioned by He et al. (2016); Chen et al. (2021), supervised machine learning algorithms usually outperform in anomaly detection use cases.

The following three common techniques are statistical analysis, time series models, and log parsing. We find three major statistical analysis techniques: descriptive analysis, outlier detection, and exploratory analysis. We categorize the technique of a project as descriptive analysis if it analyzes the data numerically, and we group it as exploratory if it uses visualization to analyze the data. Usually, the exploratory analysis will be done after performing the descriptive analysis.

As most of the input data is time series (e.g., logs, performance metrics, network traffic data), it is not surprising that some projects use time series techniques to model theirdata.

Table 4 Different types of input data, their definitions, and examples

Input data	Definition	Repository example
Monitoring Data	Different types of data that record the runtime status of a system.	
Log	System-generated data that records runtime events that have happened.	Repository (58811148) uses log data as input to perform anomaly detection.
Performance metric	Quantitative measurements used to track the performance of a system. These metrics are often operational, such as CPU usage.	Repository (160285839) uses different performance metrics such as CPU and memory usage to perform various tasks such as anomaly detection and time series forecasting.
Network-traffic data	Monitoring data that records network activities.	Repository (79239275) uses network-traffic data to identify malicious behaviors and attacks.
KPI	Measurements related to key business goals of a system. These metrics are often strategic.	Repository (142442484) uses different KPIs in time intervals to identify impactful system problems.
IoT sensor data	Data collected by devices connected to an IoT network.	Repository (142325304) uses real-time IoT sensor data to detect anomalies.
Trace	A specialized use of logging to record information about a system's execution with comprehensive details.	Repository (397983735) gathers a dataset of traces that can be used to analyze operations problems.
Multi-media	Different types of multi-media data, including images, videos, and audios.	Repository (287642401) generates summaries of data types including image data for monitoring purposes
Codebase	Source code and configuration files of software systems.	Repository (201529303) can be installed on Kubernetes source code and provide self-monitoring and self-healing.
Technical Q/A data	Data collected from technical Q/A websites such as Stack Overflow.	Repository (345320486) extracts information from Stack Overflow to find fast solutions for faults in their platform.
Alarms	Alarms generated during system run time.	Repository (160285839) uses alarmsto find the association rules between them.

To find a GitHub repository with its ID, one can either click the hyperlink or use the link https://api.github.com/repositories/*{ID}* where *{ID}* is replaced by a specific repository ID

Table 5 Different types of analysis techniques, their definitions, and examples

Analysis techniques	Definition	Repository example
Classic machine learning	Classic machine learning techniques such as Logistic Regression and Decision Tree.	
Supervised-learning	A leaning technique that uses labeled datasets.	Repository (165321356) uses various classic supervised approaches such as Random Forest and Decision Tree to detect anomalies in KPI data.
Unsupervised-learning	An approach that system learns without using labeled datasets.	Repository (58811148) implements multiple unsupervised approaches such as Isolation Forest and Invariant Miner to detect anomalies.
Deep learning	A subfield of machine learning that uses artificial neural networks with multiple layers (i.e., deep neural networks).	
Supervised-learning	A leaning technique that uses labeled datasets.	Repository (187774599) uses Artificial Neural Network to find anomalous behavior on IoT sensor data.
Unsupervised-learning	A technique that learns patterns from unlabelled data.	Repository (246569386) uses Long Short-Term Memory to detect anomalies from log data.
Statistical analysis	Analytical techniques to understand, analyze and interpret the input data.	
Descriptive analysis	Describing the features of data and summarize data in a quantitatively manner.	Repository (114942949) uses descriptive analysis such as measuring minimum and maximum values and plotting scatter figures to interpret their IoT sensor data.
Statistical outlier detection	Applying statistical tests such as z-score to identify outlier values.	Repository (156308650) uses z-score test to detect anomalies on CPU usage data.
Exploratory analysis	Exploring data to identify new connections, inspect missing data, or check hypotheses.	Repository (123162193) uses exploratory analysis such as plotting and describing their log data to inspect and understand their input data.
Time series model	Techniques that aim to model time series data, mainly used for finding trends and forecasting.	Repository (302842095) applies different time series models such as Auto Regression and Holt-winters to detect anomalies and analyze root causes.
Log parsing	Techniques that analyze and extract information from log data.	Repository (345575577) uses Drain to do log parsing. It then uses the extracted information to perform anomaly detection.

Table 5 continued

Analysis techniques	Definition	Repository example
Natural language processing	Techniques that aim to analyze and model text data.	Repository (316407231) leverages Bidirectional Encoder Representations from Transformers (BERT), a pre-trained language model as one of their techniques to perform anomaly prediction.
Others Unknown	Other techniques that could not be categorized in previous categories.	Repository (146802240) uses fuzzy matching to classify web queries. Projects for which we could not find any specific techniques.
N/A	Projects that only provide datasets or do not use any analysis techniques.	Repository (238914477) is a dataset of KPI data and does not use any analysis techniques.

To find a GitHub repository with its ID, one can either click the hyperlink or use the link https://api.github.com/repositories/*{ID}* where *{ID}* is replaced by a specific repository ID

AutoRegressive Integrated Moving Average (ARIMA) is the most common time-series technique followed by Exponentially Weighted Moving Average (EWMA).

As shown in Fig. 5, logs are the most-used input data in AIOps projects. To handle logs, projects either use log parsing techniques or Natural Language Processing (NLP) approaches. Regarding log parsing techniques, developers tend to write their customized version of log parsers (with 64%). After coming up with customized log parsers, the most common log parser that is used among AIOps projects is Drain (with 20%). Regarding NLP approaches, only 2% of AIOps projects use them, with Word2vec and BERT as the most common techniques. with the recent developments in the NLP field, for example generating language models such as BERT (Devlin et al. 2018) and CodeBERT (Feng et al. 2020), we believe AIOps solutions can also benefit more from NLP techniques. We discuss this point in more detail in

Goals: Anomaly detection is the most popular goal of the studied AIOps projects, followed by monitoring, anomaly prediction, root cause analysis, and AIOps infrastructure

We find 9 categories for the goals of the AIOps projects which are shown in Fig. 7 and are described in Table 6. Anomaly detection (60%) and monitoring (12%) are the most common reasons for using AIOps solutions. Anomaly prediction, root cause analysis, and AIOps infrastructure are the next main goals of our set of projects. We categorize the projects as "providing datasets" if they mainly provide public AIOps data that can be used by other practitioners or researchers. We categorize projects as "AIOps infrastructure" if they provide infrastructural support such as log parsing.

We find that for tasks such as anomaly detection and root cause analysis, developers usually use logs as their main source. However, for monitoring and anomaly prediction tasks, the main source of data is performance metrics. We also find that only 1% of projects do self-healing as their final goal. It means, in the other projects, after achieving the final goal, for example, anomaly detection, an agent (e.g., developer) should decide what to do with the founded anomalies. This not completely automated procedure will lead to a loss of time and resources. We discuss this point in more detail in Sect. 4.

Table 6 Different types of goals, their definitions, and examples

Goal	Definition	Repository example
Anomaly detection	Identifying anomalies that deviate from the normal behavior.	Repository (134266587) uses log data and analyze them to find anomalous ehavior. It then display anomalies using dynamic graphics.
Monitoring	Collecting and observing the real-time stream of data to understand system runtime status.	Repository (221989665) provides data monitoring and alerting.
Anomaly prediction	Analyzing historical data to forecast future anomalies.	Repository (169132015) uses metric data of hard drives to predict the failures in the near future.
Root cause analysis	Identifying the root causes of faults or problems.	Repository (238914477) analyzes the logs of Kubernetes containers to find the root causes of issues.
AIOps infrastructure	Providing infrastructure support or utility functions such as log parsing.	Repository (244678163) provides automated parsing of raw logs.
Providing datasets	Providing datasets to be used in other AIOps projects.	Repository (60705895) collects various system log datasets that can be used for log analysis.
Knowledge representation	Extracting and summarizing knowledge from datasets or websites.	Repository (345320486) extracts information from Stack Overflow to find fast solutions for faults in their platform.
Automated classification	Classifying different input data instance based on their similarities.	Repository (238914477) classifies web queries to find similar important information and trends.
Self healing	Conducting health checks and automatically fixing the issues.	Repository (238914477) aims to provide software systems with monitoring and self-healing.

To find a GitHub repository with its ID, one can either click the hyperlink or use the link https://api.github.com/repositories/*{ID}* where *{ID}* is replaced by a specific repository ID

The relation between the input data and the goals of the AIOps projects is shown in Fig. 8. As shown in this figure, log data and network traffic data are the most used data types for achieving the goal of anomaly detection, while performance metric data is the most used data type for anomaly prediction. Log data and performance metric data are also the most common data sources for the goal of root cause analysis.

Summary of RQ2

Logs are the most commonly used input data in the studied AIOps projects, followed by performance metrics and network-traffic data. Classical machine learning techniques are the most used analysis techniques, followed (far behind) by deep learning, statistical analysis, time series models, and log parsing. The most popular goals of the AIOps projects are anomaly detection, followed by monitoring, anomaly prediction, root cause analysis, and AIOps infrastructure.

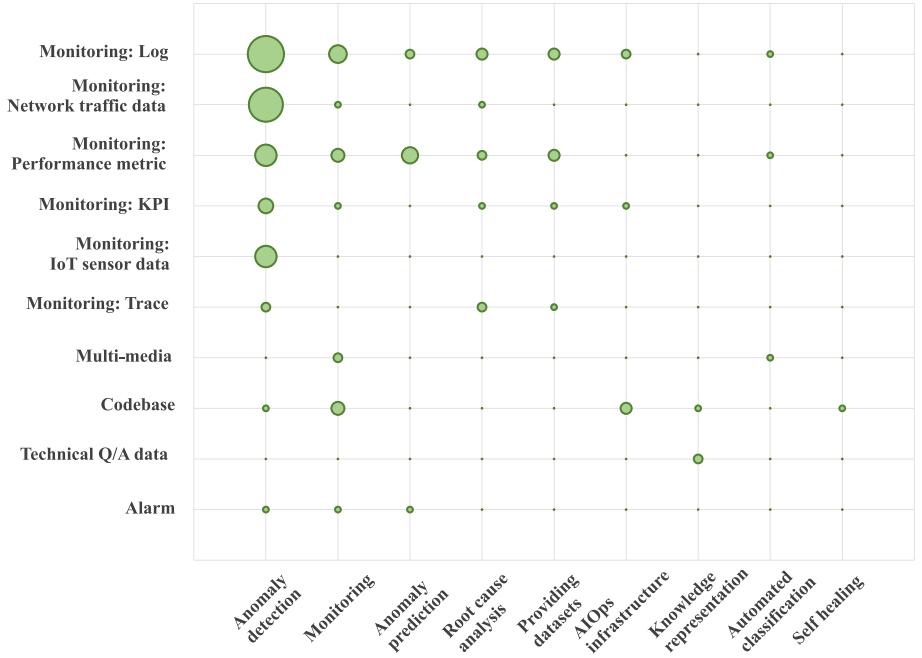


Fig. 8 The relation between the input data and goals of the AIOps projects. The sizes of the circles are proportional to the number of projects that use a certain input for a certain goal

3.3 RQ3. What is the Code Quality of AIOps Projects?

3.3.1 Motivation

about the health of the IT infrastructure...etc.. Ensuring the quality of AIOps projects is vital for the software operation tasks they are designed for. Otherwise, the insights derived from their analysis would not be reliable. Thus, in this RQ, we aim to understand the code quality of the AIOps projects. We analyze different metrics in order to come to a comprehensive conclusion. We also compare the code quality of AIOps projects with our baselines to find similar patterns or differences. Our results can provide insights for future work to improve the quality of AIOps projects.

3.3.2 Approach

We use SonarQube,[4] a static code analysis tool that supports a large number of programming languages, including Python, Java, JavaScript, and Go, to statically measure the quality of the AIOps projects and the baselines, as AIOps projects are developed by different languages (RQ1). Many recent studies have utilized or evaluated SonarQube for code quality measurement (Businge et al. 2019; Lomio et al. 2021; Saarimaki et al. 2019).

Code quality metrics To understand the code quality of AIOps projects, we measure the metrics of each studied project along four dimensions: size, reliability, maintainability, and security. The detailed list of the measured metrics include: Lines of Code (LOC), comment

[4] https://www.sonarqube.org/

Table 7 The code quality metrics and their definitions

Metric	Definition
Size	The metrics that represents the size of a project.
Lines of Code (LOC)	Number of lines that contain at least one character which is not a whitespace a tabulation, or part of a comment
Comment lines	Number of lines containing comments. Non-significant comments (empty comment lines, comment lines only having special characters, etc.) are not considered
Density of comments	The amount of lines of comments compared to lines of code. It is calculated based on the following formula Density of comments = Comment lines / (Lines of code + Comment lines) * 100
Reliability	The issues that make the code behave differently as the developer was intended.
Number of bugs	Total number of bugs in a project. A bug is defined as an issue that represents something wrong in the code
Maintainability	The issues that make the code more difficult to update than it should.
Number of code smells	Total number of code smells in a project. A code smell is a violation of design patterns that may negatively impact software quality. Rasool and Arshad (2015)
Technical debt time	The estimated time required to fix all the code smells. If the values are written in days, an 8-hour day is assumed
Security	The issues that make potential weaknesses in terms of security. These issues might be benefited by hackers.
Number of vulnerabilities	Total number of vulnerabilities in a project. A vulnebarity is a piece of code that could be exploit by a hacker
Number of security hotspots	Total number of security hotspots in a project. A security hotspot is the pieces of code that are security-sensitive

All the definitions are extracted from SonarQube. A more detailed definition could be found on SonarQube documentations

lines, density of comments, number of bugs, number of code smells, number of vulnerabilities, number of security hotspots, and technical debt time. All these quality metrics are extracted using SonarQube. We define these metrics in Table 7.

SonarQube also assigns a severity level to each of the issues. It categorizes the severities into four groups; Minor, Major, Critical, and Blocker, from the lowest to the highest severity.[5]

Finding the most violated rules and rule categories To provide insights into the primary code quality issues among the AIOps projects, we count each issue's assigned *rules* and *rule categories* (i.e., *tags*) and report the most repeated ones. SonarQube evaluates the source code against its set of rules to detect specific issues. Also, rule categories are a way to categorize trivial rules into higher-level concepts. SonarQube rules include code smells, bugs, vulnerabilities, and security hotspots. Each rule is related to a specific defined issue, and different rules can be part of one rule category.

To find the most violated issues (i.e., rules and rule categories), it is essential to consider two aspects: the occurrence of an issue in each project and the percentage of projects that have that issue. Hence, we calculate the weight of violated issues ("W" in Tables 9 and 10) to find the most violated issues. Each project can have multiple violated issues. We calculate the weight of issue i in project j using the following formula,

$$w_{ij} = \frac{n_{ij}}{n_{pj}}$$

where n_{ij} is the frequency of issue i in project j and n_{pj} is the total number of issues in project j. Then, to calculate the weight of issue i in all the projects, we calculate its average in all the projects.

$$w_i = \frac{\sum_{j=1}^{n} w_{ij}}{n}$$

Preparing project data and running SonarQube We execute the following three steps to analyze the quality of each of the AIOps and baseline projects. preprocessing needed for them, need to mention it); 3) Execute SonarQube (may briefly explain some setup/configuration).

Step 1. Clone the projects from GitHub. We clone the GitHub repositories on the local machine so that we can analyze their source code.

Step 2. Preprocess the project data. For two of the popular languages among our projects (i.e., Python and Java), we have to perform a preprocessing phase before performing the source code analysis using SonarQube. We do not need to perform this phase for other languages.

Python code. SonarQube does not support the *.ipynb* format which is the file format of Jupyter notebooks. To overcome this limitation, we first convert the *.ipynb* files in each project to the *.py* format using the *nbconvert* library (Jupyter 2022) in Python. We then use SonarQube to analyze the resulting *.py* files.

Java code. To analyze the Java code, SonarQube requires the compiled files (*.class* files). However, in most cases, developers do not upload the *.class* files in their GitHub repositories and only put the *.java* files. To address this issue, we compile the *.java* files into *.class* files. We primarily leverage the build automation tool (i.e., Gradle, Apache Maven, and Apache ANT) of the project to compile the project files. For example, for Maven projects, we use *mvn compile*. If we could not find a build automation tool, we use *javac* to compile the *.java* files.

[5] https://docs.sonarqube.org/latest/user-guide/rules/

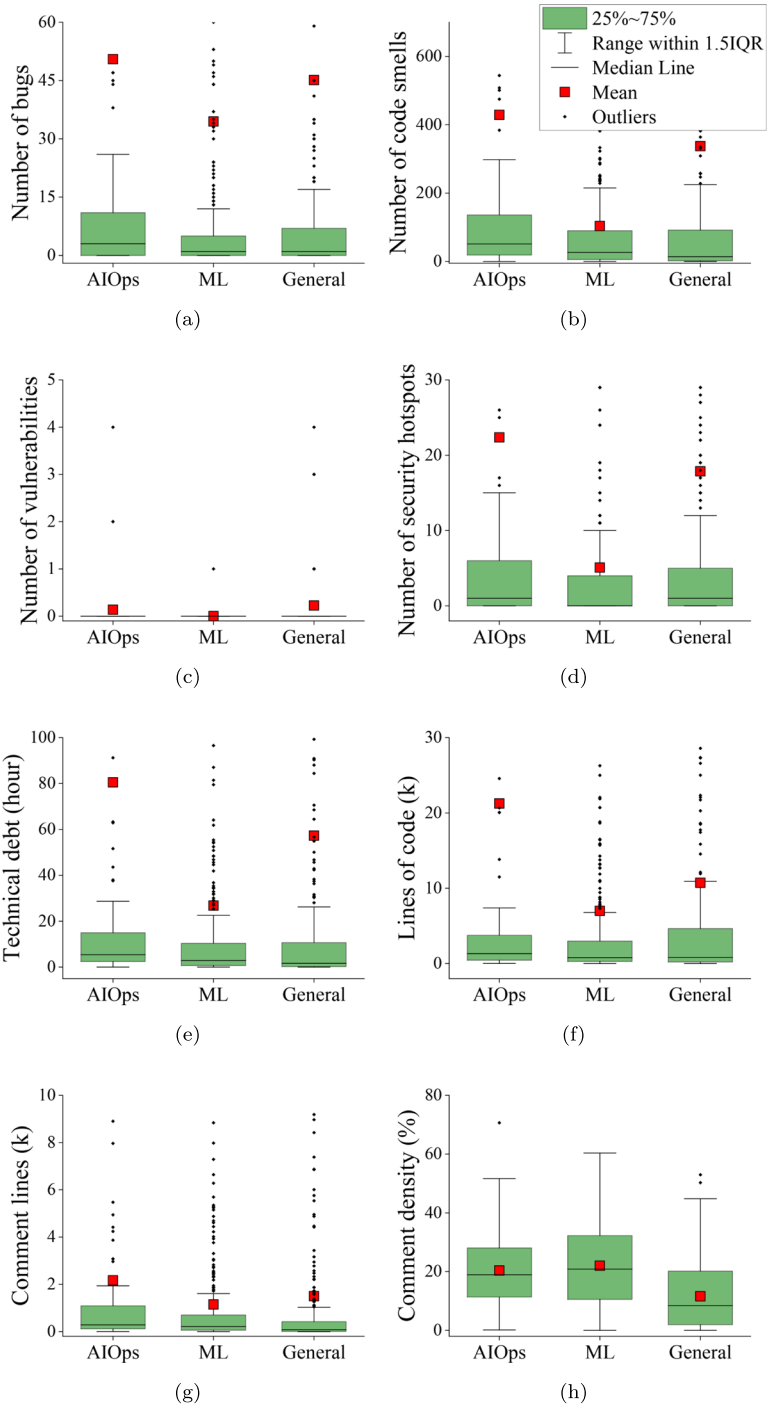


Fig. 9 Box plots of code quality metrics for AIOps and baseline projects

Step 3. Execute SonarQube. We write a script that sends the source code of each project to a SonarQube web server that is installed locally. After SonarQube finishes the analysis, we extract the results.

We are able to analyze the source code of 770 out of the 887 projects (87%) in all three groups of projects (AIOps projects and the two baselines). The success ratio is 93%, 92%, and 79% for AIOps, ML, and General projects, respectively. The main reasons of failures include removed GitHub repositories and failures in compiling the Java projects. Analyzing the source code of projects at this scale is rarely reported in the literature, and performing the SonarQube analysis experiments took several weeks.

Statistical tests Similar to the first RQ, we perform statistical tests to validate our results and ensure their statistical significance. We use *Mann–Whitney U* test (Mann and Whitney 1947) to compare the distribution of the metrics between AIOps and baseline projects. We also use *Cliff's delta* test (Cliff 1993) to measure the effect size of the difference between the distributions. We use the same approach and scale (i.e., effect of |d| = 0.147 is small, |d| = 0.33 is medium, and |d| = 0.474 is large) as described in Sect. 3.1.2.

3.3.3 Results

AIOps projects have poorer quality than the ML and General baselines Fig. 9 represents the box plots of code quality metrics extracted from SonarQube for AIOps and baseline projects. Table 8 illustrates the p-value and effect size of the code quality metrics of AIOps projects comparing to the baselines. For bugs, code smells, vulnerabilities, security hotspots, and technical debt, we further report the values normalized by the LOC of each project. We report these normalized values in Fig. 10 to ensure that our findings are robust and not biased by the size of the projects.

According to the p-values of *Mann–Whitney U* test shown in Table 8, the difference between most of the code quality metrics of AIOps projects compared to baselines is significant. Considering the number of bugs and code smells, AIOps projects are statistically different from the baselines, and the effect size of their differences is small or negligible. As shown in Fig. 9a, AIOps projects have a higher number of bugs in terms of both mean and median, with a median of 3 bugs, while the median is 1 in both baselines. This pattern is also

Table 8 Detailed results of *Mann–Whitney U* and *Cliff's delta* tests on projects' code quality

Metric	AIOps vs. ML		AIOps vs. General	
	p-value	effect size	p-value	effect size
Bugs	0.01	**	0.03	*
Code smells	0.00	**	0.00	**
Vulnerabilities	0.00	*	1.00	-
Security hotspots	0.03	*	0.18	-
Technical debt	0.00	**	x0.00	**
LOC	0.01	**	0.02	*
Comment lines	0.01	**	0.00	***
Comment density	0.42	-	0.00	***

Mann–Whitney U results are shown in *p-value* columns. If the sets have statistically different distributions, the *Cliff's delta* results are shown in *effect size* columns.
*: negligible effect **: small effect ***: medium effect

seen in the normalized version in Fig. 10a. The number of code smells is also higher in AIOps projects based on both original and normalized versions presented in Figs. 9b and 10b. The median number of code smells in AIOps projects is 51.0, twice the ML baselines with 26.5 and more than 3.5 times the General baseline with 14.0.

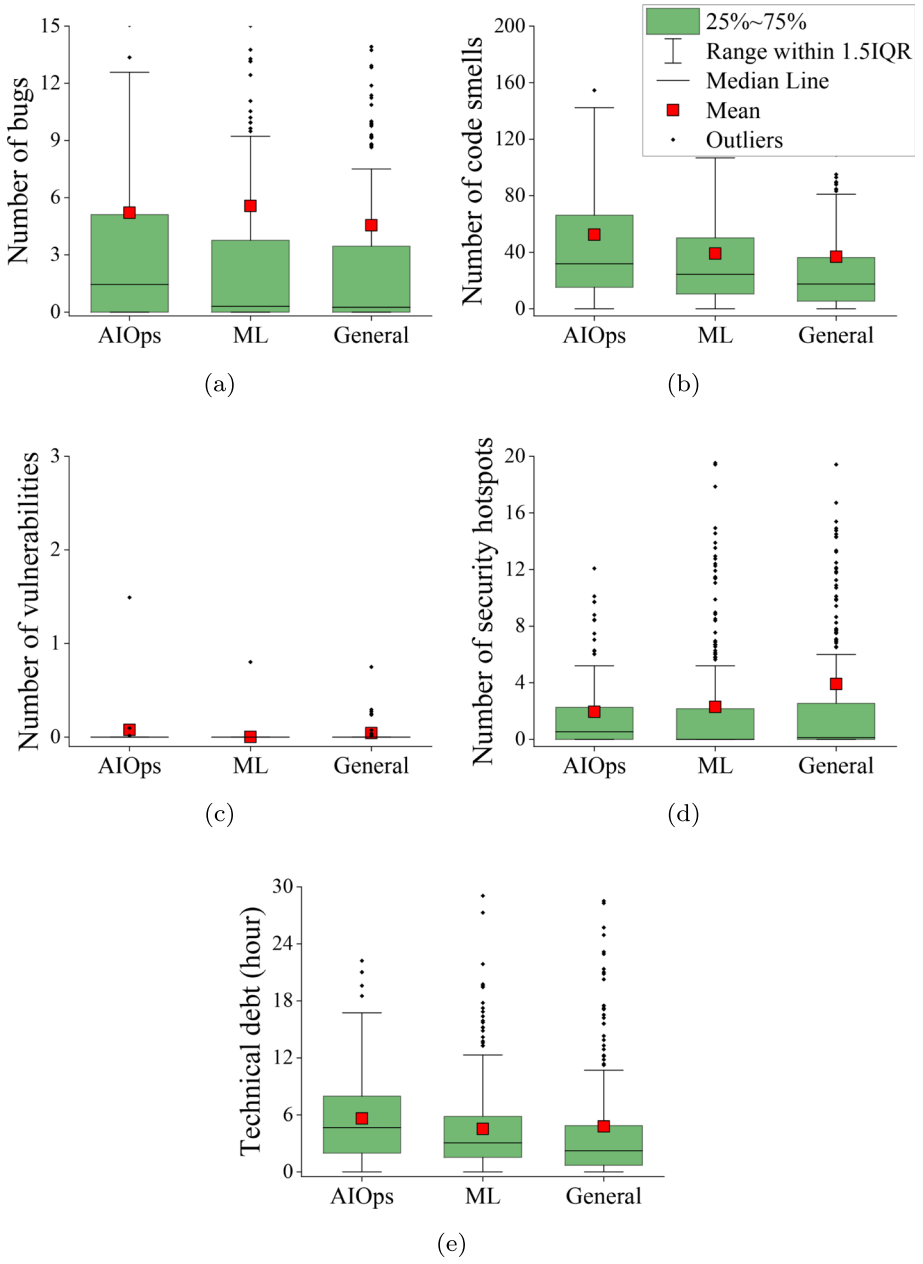


Fig. 10 Box plots of code quality metrics for AIOps and baseline projects. The values are normalized by LOC of each project

Regarding the number of vulnerabilities and security hotspots among the projects, no statistically significant difference is seen between AIOps and General projects, according to Table 8. However, AIOps and ML baseline are statistically different, with an effect size of negligible. As Figs. 9c and 10c show, most of the AIOps and baseline projects do not have any vulnerabilities. However, as Fig. 9d represents, the median number of security hotspots in AIOps is 1 while it is 0 in ML baseline; and the mean number of security hotspots in AIOps is 22.4 while it is only 5.1 in ML baseline.

Technical debt is the next analyzed security metric. Table 8 suggests that there is a statistically significant difference between AIOps and baselines with a small effect size. As Fig. 9e shows, both the mean and median of AIOps technical debt are higher than the baselines. The mean value of technical debt for the AIOps set is 80.5 h, while it is 26.8 and 57.3 h for ML and General baselines, respectively. The median value of technical debt in AIOps (5.4 h) is 2 times the ML baseline (2.9 h) and 3 times the General baseline (1.7 h). As Fig. 10e suggests, technical debt in AIOps projects is higher than the baselines, even in the normalized version. These findings are in line with the high volume of issues observed in AIOps repositories, as shown in Fig. 3.

AIOps projects have more lines of code than baselines and the same amount of comments as the ML baseline Table 8 shows that AIOps set is statistically different from the baselines in terms of number of lines of code and number of comment lines. According to Fig. 9f, the mean value of LOC for AIOps projects is 21.2k, 3 times the ML baseline (with 7.0k) and 2 times the General baseline (10.7k). The median value also indicates a similar pattern, with 1.4k for AIOps projects and 0.8k for the baselines. The amount of comments written in AIOps projects is also higher than the baselines, with a median of 0.3k compared to 0.2k and 0.1k for the ML and General baselines. Figure 9h shows the comment density between the three sets. The density of comments in AIOps and ML projects are similar, with the medians of 19.0% and 20.9%, respectively. The comment density for the General baseline is less than half of the AIOps set, with only 8.4%.

AIOps projects suffer from code smells more than other types of issues Fig. 11 illustrates the distribution of bugs, code smells, vulnerabilities, and security hotspots among the projects. The percentage of vulnerabilities is much smaller than other types of issues in all the projects. In AIOps projects, the main issue is related to code smells, which account for 85% of all issues, 13% more than the ML baseline. After code smells, the most common type of issue is bugs, with 10%, and then security hotspots, with 5%.

AIOps and ML projects have similar types of issues in terms of violated SonarQube rules and rule categories We report the most violated rules and rule categories in Tables 9 and 10. We also define these violated rules and rule categories in AIOps field (based on SonarQube website[6][7]) in Tables 11 and 12.

As shown in the tables, AIOps and ML projects have similar rule and rule category violations. In terms of violated rules, 9 out of the top-10 violated rules are common between AIOps and ML projects. Furthermore, the first two most violated rules (i.e., python:S117 and python:S125) are the same in AIOps and ML projects, with similar weights (17.6% and 14.6% for AIOps, and 16.8% and 15.5% for ML projects). Also, in terms of most violated

[6] https://rules.sonarsource.com/

[7] https://docs.sonarqube.org/latest/user-guide/built-in-rule-tags/

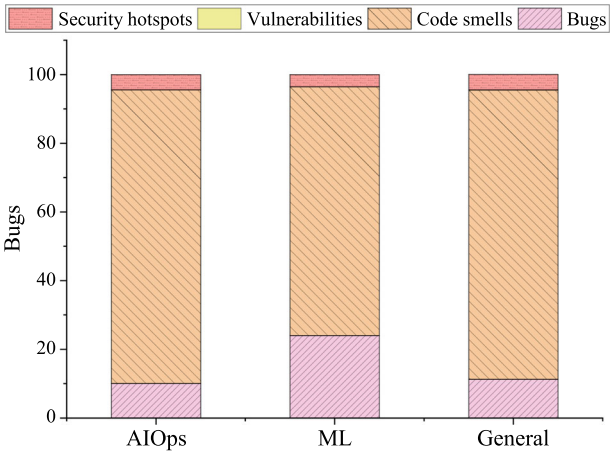


Fig. 11 The distribution of the quality issues among the AIOps and baseline projects

rule categories, 9 out of the top-10 violated rule categories are common between AIOps and ML projects.

Comparing issues between the AIOps projects and the General baseline, 4 of the top-10 violated rules and 7 of the top-10 violated rule tags are common. However, the weight and percentage of projects having these issues in AIOps projects are much higher. As an example, Python:S117 is the most violated rule in both AIOps and General projects. However, the weight and percentage of projects having this violated rule in AIOps are 17.6% and 59.1% but in the General baseline are 5.2% and 19.7%, respectively.

We provide the complete list of violated rules and rule categories in AIOps projects and other baselines in our replication package.

Naming convention, commented-out code, duplicated string literals, high complexity of functions, and wildcard imports are among the top violated rules in AIOps projects

Regarding the most violated rules in AIOps, the first three are *python:S117*, *python:S125*, and *python:S1192*. The first one has a minor severity and is about the non compliance of naming convention. Shared naming conventions are vital and allow teams to collaborate effectively. The second one has a major severity and is about commenting out the unused sections of code which reduces readability. Instead, unused code should be deleted. The third one has a critical severity and is about using duplicated string literals. It makes the process of refactoring error-prone because the programmer must be sure to update all occurrences of the string.

In addition to these violated rules, three more rules with critical severity exist in the top 10 violated rules of AIOps projects; *python:S3776*, *python:S2208*, and *python:S5754*. *Python:S3776* is about the cognitive complexity of code. It measures how hard the control flow of a function is to understand. Functions with high cognitive complexity will be difficult to maintain. *Python:S2208* is about using wildcard imports (i.e., from module import *). Importing all public names from a module has multiple disadvantages. It can lead to conflicts between local names and imported ones, or same names between two different packages. It also reduces code readability and may cause confusion about which classes are imported

Table 9 The top-10 violated SonarQube rules for AIOps projects and the baselines. "Sev" indicates the severity of issues, "W" represents the weight of rules, and "N" is the percentage of projects with that rule. "Mn" stands for Minor, "Mj" stands for Major, "Cr" stands for Critical, and "Bl" stands for Blocker

AIOps				ML				General			
Rule	Sev	W(%)	N(%)	Rule	Sev	W(%)	N(%)	Rule	Sev	W(%)	N(%)
python:S117	Mn	17.6	59.1	python:S117	Mn	16.8	59.3	python:S117	Mn	5.2	19.7
python:S125	Mj	14.6	66.3	python:S125	Mj	15.5	63.8	python:S125	Mj	3.9	20.3
python:S1192	Cr	7.8	67.3	python:S905	Mj	7.7	27.7	javascript:S1117	Mj	3.4	19.3
python:S905	Mj	7.0	25.5	python:S1192	Cr	7.3	55.4	Web:S5254	Mj	3.0	20.7
python:S1481	Mn	4.2	53.6	python:S1481	Mn	5.9	52.5	python:S1192	Cr	2.6	19.0
python:S3776	Cr	3.3	48.1	python:S1542	Mj	4.0	41.0	javascript:S125	Mj	2.3	16.7
python:S1542	Mj	3.2	44.5	python:S3776	Cr	3.2	43.5	python:S1481	Mn	1.9	18.0
python:S2208	Cr	2.3	25.4	python:S2320	Mj	2.8	8.8	xml:S125	Mj	1.8	5.2
python:S5754	Cr	2.2	35.4	python:S2208	Cr	2.4	23.4	Web:S1827	Mj	1.7	7.5
python:S100	Mn	2.1	20.9	python:S5754	Cr	2.1	25.4	javascript:S2703	Bl	1.7	12.4

Table 10 The top-10 violated SonarQube tags (rule categories) for AIOps projects and the baselines. "W" represents the weight of tags, and "N" is the percentage of projects with that tag

AIOps			ML			General		
Tag	W(%)	N(%)	Tag	W(%)	N(%)	Tag	W(%)	N(%)
convention	25.7	76.4	unused	26.7	83.6	unused	15.1	62.3
unused	24.2	93.6	convention	21.9	67.8	convention	10.8	40.0
design	9.1	79.1	cwe	6.7	56.0	suspicious	7.5	52.5
cwe	8.3	66.4	design	6.5	57.6	pitfall	7.5	47.9
suspicious	5.6	66.4	suspicious	4.4	48.0	cwe	6.1	50.5
brain-overload	4.2	60.9	pitfall	3.4	39.8	design	5.1	38.4
pitfall	3.3	42.7	brain-overload	3.3	49.2	accessibility	4.5	29.2
error-handling	2.1	49.1	obsolete	2.0	13.6	brain-overload	3.6	43.6
bad-practice	2.0	51.8	bad-practice	2.0	37.9	wcag2-a	3.5	27.5
clumsy	1.6	36.4	error-handling	2.0	33.6	confusing	3.1	38.0

Table 11 The most violated SonarQube rules and their definitions in AIOps projects

Rules Name	Definition
python:S117	Local variable and function parameter names should comply with a naming convention
python:S125	Sections of code should not be commented out
python:S1192	String literals should not be duplicated
python:S905	Non-empty statements should change control flow or have at least one side-effect
python:S1481	Unused local variables should be removed
python:S3776	Cognitive Complexity of functions should not be too high
python:S1542	Function names should comply with a naming convention
python:S2208	Wildcard imports should not be used
python:S5754	*SystemExit* exception should be re-raised immediately
python:S100	Method names should comply with a naming convention

and used. *python:S5754* is about handling exceptions. This rule indicates that *SystemExit* exception should be re-raised immediately.

Naming convention, unused or commented-out code, and poor design are among the top violated rule categories Regarding the most violated rule categories in AIOps repositories, *convention*, *unused*, and *design* are the worst issues. *Convention* category is about

Table 12 The most violated SonarQube tags (rule categories) and their definitions in AIOps projects

Tags Name	Definition
convention	Coding convention - typically formatting, naming, whitespace
unused	Unused code; e.g., a private variable that is never used
design	There is something questionable about the design of the code
cwe	Relates to a rule in the Common Weakness Enumeration For more information, visit https://cwe.mitre.org/
suspicious	It's not guaranteed that this is a bug, but it looks suspiciously like one. At the very least, the code should be re-examined and likely refactored for clarity
brain-overload	There is too much to keep in your head at one time
pitfall	Nothing is wrong yet, but something could go wrong in the future; a trap has been set for the next person, and they'll probably fall into it and screw up the code
error-handling	Issues related to handling the errors such as *Exception* methods
bad-practice	The code likely works as designed, but the way it was designed is widely recognized as being a bad idea
clumsy	Extra steps are used to accomplish something that could be done more clearly and concisely

fulfilling coding conventions, including naming functions and variables, complying white-spaces and indentations. The second most violated rule category is *unused*. This category is about unused code that decreases the performance of the system. Some examples of this category are unused assignments, unused private classes, and empty test cases. The third most ignored rule category, *design*, is about the bad design of the software. Some examples of this group are unstable tests, duplicated string literals, and using randomized data in test cases.

Having rule category issues such as naming conventions, unused code, and bad design indicate that developers tend to write AIOps code in an *ad hoc* manner, which hinders their project's reusability and maintainability. Also, looking at the most violated rules and rule categories, we realize paying more attention to details can increase the quality of AIOps solutions. For example, 3 out of the top-10 violated rules are related to naming conventions. Only following the naming conventions can reduce the number of issues heavily. Furthermore, since most of the rules (9 of top-10) and rule categories (9 of top-10) are common in AIOps and ML projects, AIOps projects can benefit from the quality assessment and quality assurance tools and techniques that have been developed for ML systems. The above points are discussed in more detail in Sect. 4.

Summary of RQ3

Although AIOps projects have an adequate amount of comments compared to the ML and General baselines, AIOps projects exhibit poorer quality than the two baselines in terms of a variety of quality metrics (e.g., bugs, code smells, and technical debt). In particular, code smells are the dominant type of issue in AIOps projects. We also identify the most common issues in AIOps projects: naming convention, unused code, and bad design are the top-3 violated rule categories. Moreover, we observe the similarity of violated rules between AIOps and ML projects. Future efforts are needed to reduce the issues identified in this work and improve the quality of AIOps projects, e.g., by designing tools for fixing bugs/smells or deriving coding guidelines. We also encourage future AIOps projects to reduce the *ad-hoc*ness of their code to improve reusability and maintainability.

4 Discussion

Based on the findings of our study, in this section, we discuss the state of AIOps in open-source projects. We then discuss the current challenges of developing AIOps applications and foresee possible future directions for AIOps researchers and practitioners. Finally, to assess the robustness of our findings, we add a stricter filtering criterion and report the main results considering only more mature projects. We compare the results obtained from these projects with our main findings to make sure our findings are robust enough.

4.1 AIOps: Where is it Now?

The number of AIOps applications is growing fast, and they are receiving a lot of attention from the open-source community. Regarding the results of Sect. 3.1.3, we find that the speed of growth for AIOps applications is faster than that for general-purposed

projects on GitHub. We further find that AIOps applications are receiving more attention regarding GitHub metrics compared to machine learning and general-purposed applications. This attention towards the AIOps area encourages researchers and practitioners to develop new technical innovations in various areas of AIOps. We discuss some of the potential future directions below.

Monitoring data-especially logs, performance metrics, and network-traffic data-are the most common data types of AIOps applications Regarding the results of Sect. 3.2.3, we find that almost 70% of the projects use logs, performance metrics, or network-traffic data. The usage of other types of input data (e.g., alarms) is limited in open-source AIOps applications. This finding is in line with Notaro et al. (2021b) that many AIOps-related papers also use event logs and performance metrics as their data sources. Future efforts may be invested to investigate how to optimally process these common data types in various downstream tasks. For example, time-series data representation techniques (Wilson 2017) and time-series segmentation techniques (Lovrić et al. 2014) may be explored to improve data representations of such time-series data.

Classical machine learning is the most common technique in AIOps applications We find that over half (54%) of the AIOps applications use classical machine learning as their primary technique, while only 14% use deep learning techniques. AIOps practitioners also develop AIOps applications using a variety of methods, including time-series and statistical models. However, some techniques (e.g., natural language processing) are not used frequently. We suggest that future AIOps solutions may further utilize more sophisticated techniques of deep learning and natural language processing. However, as suggested by Li et al. (2020b); Lyu et al. (2021), AIOps models should be trustable and interpretable; hence, using black-box models without the ability to interpret is not recommended. Therefore, we also suggest that future AIOps solutions should always experiment with classical solutions instead of simply assuming deep learning techniques are the optimal solution.

Anomaly detection is the most common goal of AIOps applications 60% of open-source AIOps applications' primary goal is to detect anomalies. Monitoring, anomaly prediction, root cause analysis, and providing AIOps infrastructure are the other common goals of AIOps applications. Only 5% of projects provide a public AIOps dataset, and only 1% of projects provide self-healing features. Our results align with the findings of Notaro et al. (2021b), where they observe that 62% of AIOps papers are associated with failure management, including failure detection, failure prediction, and root cause analysis.

AIOps applications suffer from more code quality issues than machine learning and general-purpose-based projects Considering our results in Sect. 3.3.3, AIOps applications have a higher number of quality issues (e.g., bugs and code smells) compared to the baselines. We list the most common violated issues in AIOps applications in Tables 9 and 10. AIOps practitioners may pay attention to these issues in their applications and address them to reduce the risk of future problems.

4.2 AIOps: Challenges and Future Directions.

More benchmarking datasets could be designed for AIOps applications. We find that only 5% of the projects aim to design and publish AIOps benchmark datasets. As also mentioned

by Bogatinovski et al. (2021), there is a lack of good and public AIOps benchmarks, where different AIOps approaches could be compared. As most of the AIOps applications use logs, performance metrics, and network-traffic data as their input data source, we suggest AIOps researchers design and publish real-world and public datasets of these data types so other AIOps practitioners and researchers could leverage them in their applications. In the future, we also plan to design a benchmarking framework that can help future work generate customized datasets.

Natural Language Processing (NLP) techniques can receive more attention in AIOps in the coming years Our results in Sect. 3.2 indicate that only 2% of the studied projects use NLP techniques. On the other hand, approximately one-third of the projects use logs as their input data. Prior research has shown that logs can be approximately represented as natural language text since the logs are generated by logging statements in the source code written by humans (Hindle et al. 2016). Other works also illustrate that logs are even more predictable and repetitive than natural languages, such as English (He et al. 2018a; Tu et al. 2014; Yao et al. 2020). Hence, we suggest that AIOps projects can benefit from leveraging the advances in NLP techniques to analyze and model the input data such as logs.

More attention should be paid to increasing automation and reducing human interventions in AIOps solutions Regarding the goal of the studied projects, most projects aim to detect and predict anomalies, monitor, and analyze the root causes of failures. All these systems need human interventions when the goal is reached. For example, when an anomaly is detected, an operator should decide the next required action to handle the situation. As demonstrated in Fig. 7, only 1% of projects aim for self-healing, meaning that the system can make necessary changes when needed without human interventions. We believe AIOps solutions should move toward becoming more automated, detecting/predicting the incidents and resolving them autonomously. Self-healing techniques may get more attention and be integrated with detection/prediction techniques (Ding et al. 2012, 2014; Lou et al. 2017).

Paying more attention to simple details can increase the quality of AIOps solutions We identify the most common issues in AIOps projects in Sect. 3.3.3. When looking at the most violated rules, some of them are challenging and time-consuming to fix and need structural changes in code, such as *python:S3776* (cognitive complexity of functions) and *python:S1192* (duplication of string literals). On the other hand, some are easy to handle. 3 of the 10 most violated rules are related to naming conventions. Only following the naming conventions can reduce the number of issues heavily. Furthermore, rules such as *python:S2208* (using wildcard imports) and *python:S1481* (removing unused variables) are quick to determine and easy to fix. Identifying the most common issues help practitioners and researchers in the field to become aware of them and take measurements to reduce them.

Most of the tools and techniques used for the quality assurance of machine learning systems can also be used for AIOps solutions Our results indicate that a high proportion of issues in ML and AIOps techniques are the same (90% of most violated rules and 90% of most violated rule categories are shared). Besides, the quality assurance of ML systems has received special attention in the previous years, and different approaches have been built to preserve the quality of ML-based systems (Braiek and Khomh 2022; Nakajima 2018; Nikanjam et al. 2021; Poth et al. 2020; Tambon et al. 2023). Hence, we believe most of the

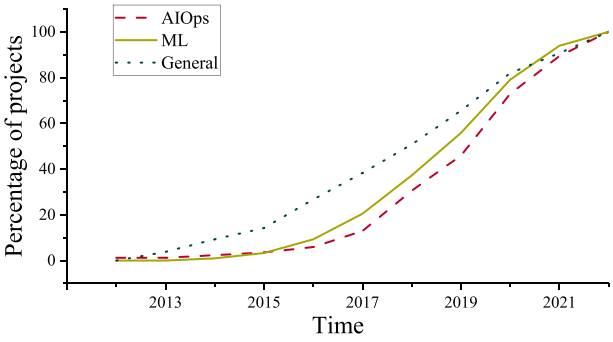


Fig. 12 The cumulative distribution of the creation time of AIOps and baseline projects with the new filtering criterion

developed tools and techniques for quality assurance in ML systems can be utilized in AIOps techniques. Applying these tools and techniques may reduce the quality issues mentioned in Sect. 3.3.3.

Open-source AIOps solutions can benefit from broader use-cases and scenarios As described in previous works (Dang et al. 2019; Prasad and Rich 2018), AIOps can encompass a wide range of scenarios, from predicting the future status of systems to improving the productivity of engineers. However, the scope of current AIOps projects available on GitHub appears to be limited, focusing primarily on certain aspects with a skewed distribution. For example, we find that anomaly detection is the dominant goal of AIOps projects with 60% of use cases. To address this disparity, AIOps researchers and practitioners may pay more attention to other AIOps scenarios, such as minimizing engineers' tedious tasks and better system automation. This can contribute to advancing this interdisciplinary field and realizing its full potential across diverse use cases and applications.

4.3 The Impact of Using a Stronger Filtering Criterion

To assess the robustness of our findings, we add an extra filter to both AIOps and baseline projects and discuss the main findings of RQ1 and RQ3. The intention is to confirm that the obtained results are meaningful insights and are robust. To do so, we analyze the distribution of the AIOps projects based on their number of stars. Then, we use the elbow curve to

Table 13 The top-5 languages of AIOps and baseline projects with the new filtering criterion

AIOps Language	Usage (%)	ML Language	Usage (%)	General Language	Usage (%)
Python	65.9	Python	81.4	Python	18.6
Java	10.6	C++	2.3	JavaScript	15.3
Go	4.7	HTML	2.3	Java	7.7
HTML	3.5	JavaScript	1.9	Typescript	6.6
JavaScript	2.4	MATLAB	1.4	C	6.0

determine a threshold to filter the projects, similar to prior work which uses the elbow curve to filter the AutoML tools to be studied (Majidi et al. 2022). Specifically, we create a line chart of the sorted number of stars of the projects and find the elbow point. The elbow point is the distinct bend or "elbow" in a line chart that indicates a change in the distribution (Kodinariya et al. 2013). We find that the elbow point in the number of stars of AIOps projects is 5. Hence, we add extra filtering and only consider projects that have the number of stars greater or equal to 5 (stars: >= 5). We also add this filtering criterion to the ML and General baselines to ensure consistency. Adding this criterion reduces the number of AIOps projects from 119 to 85, ML baseline from 383 to 215, and General baseline from 385 to 183.

Results of RQ1 with the new filtering criterion We report the main results of RQ1 in regard to the new filtering criterion. Figure 12 presents the percentage of projects created in and before each year for AIOps and baseline projects. As illustrated, AIOps projects are growing faster than ML and General baselines, which is also the same trend in our initial results (cf. Figure 2).

Table 13 shows the top-5 most common languages in the AIOps and baseline sets. The dominant programming language in AIOps projects is Python, with usage of 65.9%, followed by Java with 10.6%. The most used language in the ML projects is Python with 81.4%, and

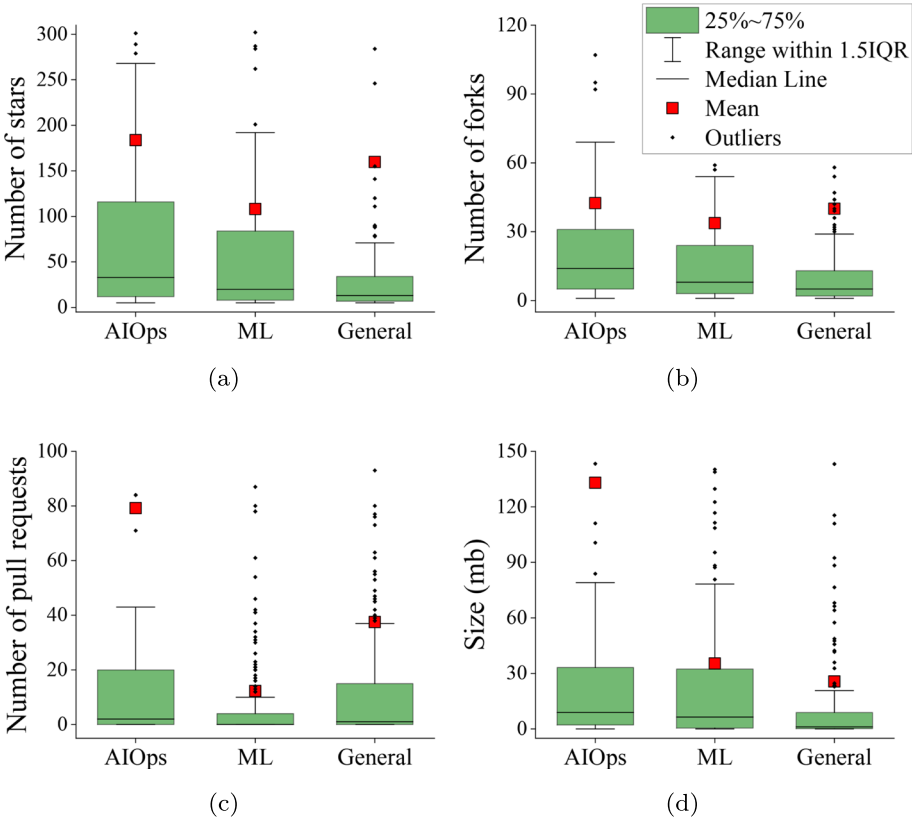


Fig. 13 Box plots of GitHub metrics for AIOps and baseline projects with the new filtering criterion

Table 14 Detailed results of *Mann–Whitney U* and *Cliff's delta* tests on projects' GitHub metrics with the new filtering criterion

Metric	AIOps vs. ML		AIOps vs. General	
	p-value	effect size	p-value	effect size
Stars	0.02	**	0.00	***
Forks	0.01	**	0.00	***
Pull requests	0.02	**	0.81	-
Size	0.14	-	0.00	***

Mann–Whitney U results are shown in *p-value* columns. If the sets have statistically different distributions, the *Cliff's delta* results are shown in *effect size* columns.
*: negligible effect **: small effect ***: medium effect

the most common language in General baseline is Python with 18.6%, followed by JavaScript with 15.3%. All these trends are also visible in our initial results (cf. Table 2).

Fig. 13 depicts the box plots of some of the GitHub metrics for the AIOps and baseline projects. Table 14 illustrates the p-value and effect size of the same metrics of AIOps projects compared to the baselines. Regarding the number of stars and forks, AIOps projects demonstrate higher popularity than the baselines, as evidenced by both the median and mean values. The median value of stars in AIOps, ML, and General sets are 33, 20, and 13, respectively. Also, the median value of forks in AIOps, ML, and General sets are 14, 8, and 5, respectively.

Table 14 shows that the number of stars and forks have a small effect size compared to the ML baseline and a medium effect size compared to the General baseline. Considering the number of pull requests, as shown in Fig. 13c, AIOps projects experience more of them, with a median value of 2, compared to 0 in ML and 1 in General. The mean value of pull requests in AIOps projects is also much higher than the baselines. Statistical tests suggest that the effect size of pull requests in AIOps compared to the ML baseline is small, and there is not a significant difference between AIOps and the General baseline. According to Fig. 13d, AIOps projects have a larger size compared to the baselines. With a median size of 9.0 MB in AIOps projects, they tend to be 9 times larger than General baselines with a median size of 1.1 MB. Statistical tests suggest that the difference between AIOps and General baseline in terms of their size is medium. However, there is no significant difference between AIOps and ML baseline. All the mentioned results and trends are consistent with our initial results (cf. Figure 3 and Table 3). In summary, when we apply a stricter project filtering criterion, our main findings in RQ1 still hold, indicating the robustness of our findings.

Results of RQ3 with the new filtering criterion We also report the main results of RQ3 with the more mature projects that are selected with the stricter filtering criterion. Figure 14 shows the box plots of some of the code quality metrics extracted from SonarQube for AIOps and baseline projects. Please check Table 7 for the definitions of these code quality metrics. Besides, Table 15 presents the p-value and effect size of the code quality metrics of AIOps projects compared to the baselines. As shown in Fig. 14, AIOps projects experience more bugs and code smells than the baselines. The median amount of bugs in AIOps is 3, while it is 1 in ML and 0 in General baselines. Besides, the median amount of code smells is 57 in AIOps, while it is 27 in ML and 9 in General baselines. Furthermore, the amount of technical debt in AIOps projects is much higher than the baselines (the median value of 6.5 h for the AIOps set, 2.9 h for the ML set, and 1.2 h for the General set). The LOC in AIOps projects is also higher than the baselines.

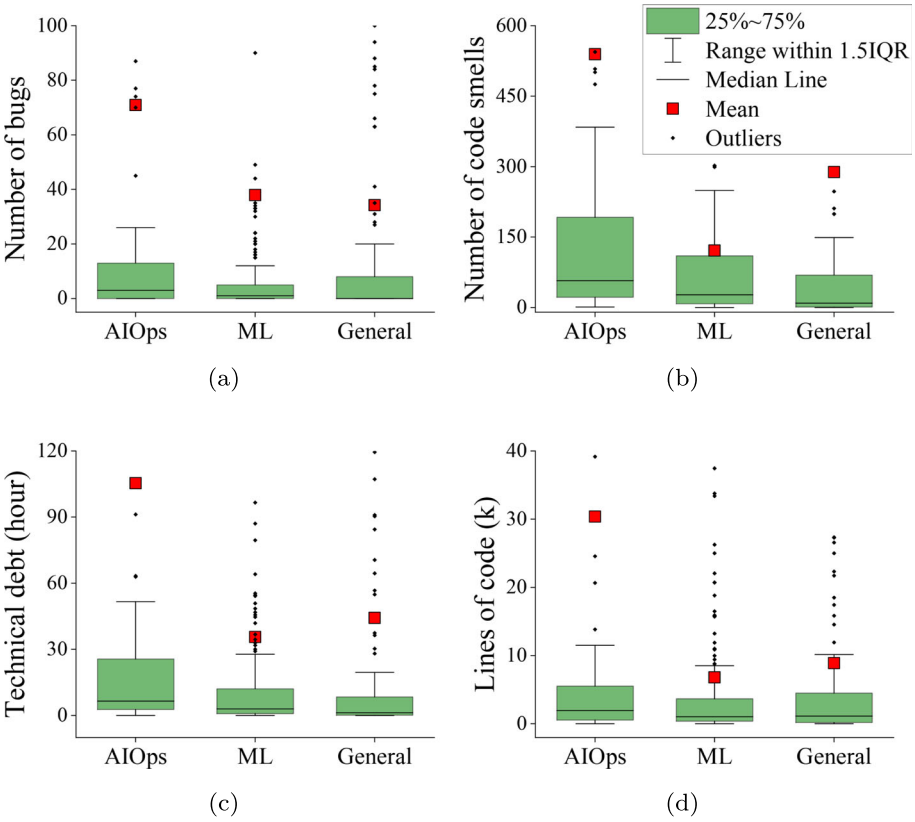


Fig. 14 Box plots of code quality metrics for AIOps and baseline projects with the new filtering criterion

While all these trends are also visible in the initial results (cf. Figure 9), it seems that the more mature AIOps projects (i.e., the projects with the new filtering criterion) have more LOC and thus more issues in terms of bugs, code smells, and technical debt.

Considering the statistical tests presented in Table 15, we can observe that AIOps projects are statistically different from both baselines in the mentioned quality metrics (i.e., number of bugs, number of code smells, technical debt, and lines of code). Besides, the effect sizes

Table 15 Detailed results of *Mann–Whitney U* and *Cliff's delta* tests on projects' code quality with the new filtering criterion

Metric	AIOps vs. ML		AIOps vs. General	
	p-value	effect size	p-value	effect size
Bugs	0.01	**	0.01	**
Code smells	0.00	**	0.00	***
Technical debt	0.00	**	0.00	***
LOC	0.02	**	0.01	**

Mann–Whitney U results are shown in *p-value* columns. If the sets have statistically different distributions, the *Cliff's delta* results are shown in *effect size* columns.
*: negligible effect **: small effect ***: medium effect

Table 16 The top-5 violated SonarQube rules and tags (rule categories) for AIOps projects and the baselines with the new filtering criterion. "W" represents the weight of tags, and "N" is the percentage of projects with that tag

AIOps			ML			General		
Rule	W(%)	N(%)	Rule	W(%)	N(%)	Rule	W(%)	N(%)
python:S117	17.1	58.7	python:S117	17.5	60.7	python:S117	3.7	15.3
python:S125	12.7	66.7	python:S125	14.4	64.2	Web:S5254	3.2	24.7
python:S1192	6.7	65.3	python:S1192	7.8	60.2	python:S125	2.9	18.0
python:S905	4.8	22.7	python:S1481	6.3	57.7	javascript:S1117	2.8	18.0
python:S1481	3.9	60.0	python:S905	5.5	23.9	Web:S1827	2.7	10.0
Tag	W(%)	N(%)	Tag	W(%)	N(%)	Tag	W(%)	N(%)
convention	25.7	76.4	unused	25.0	85.1	unused	12.1	58.0
unused	24.2	93.6	convention	22.2	69.2	convention	8.9	34.7
design	9.1	79.1	design	7.1	63.2	pitfall	7.3	44.7
cwe	8.3	66.4	cwe	5.8	59.2	suspicious	7.0	50.0
suspicious	5.6	66.4	suspicious	5.1	55.7	accessibility	4.8	32.7

of the differences are small or medium. Compared to the initial results presented in Table 8, a larger effect size can be seen while comparing AIOps and General baseline.

We also report the most violated rules and rule categories in Table 16. These rules and rule categories are defined in Tables 11 and 12, respectively. Comparing the new criterion (Table 16) with the initial criterion (Tables 9 and 10), no change is seen for the AIOps projects. The top-5 rules and rule categories remain the same, with similar weight (i.e., W(%) column in Table 16) and usage (i.e., N(%) column in Table 16). Another finding (alike to the initial results) is the similarity between AIOps and ML projects in terms of their quality issues. Overall, the results of our stricter filtering criterion show that our initial findings are robust, as the trends and main findings remain the same.

5 Threats to Validity

This section discusses threats to the validity of our results.

External validity In this work, we identified and studied a set of AIOps projects on GitHub. These projects may not cover all AIOps projects on GitHub, those hosted on other platforms, or private projects. Besides, as "AIOps" is a new terminology, not all AIOps projects mention the keyword of "AIOps" in their repositories. Future work examining other sources of AIOps projects (e.g., those published in the literature or closed-source projects) can complement our results. To broaden the generalizability of our work and maximize the coverage of AIOps projects, we follow a process that combines automated search (two rounds), keyword expansion, manual verification, and filtering. Two authors of the paper carefully examined each of the candidate projects to select the ones for our study. The third author steps in to resolve any disagreement. Nevertheless, future work can leverage our replication package and extend our study by analyzing more AIOps projects.

To select the projects, we consider repositories with stars and forks greater than or equal to 1. We selected this criteria to have a balance between filtering the low-bar projects and having a proper portion of projects to study. Having stricter filtering criteria would heavily reduce the number of AIOps projects, as there are not many AIOps projects available on GitHub. However, having low filtering criteria could potentially lead to biased results and lack of robustness in the findings. To address this concern and enhance the robustness of our findings, we conducted a sensitivity analysis (cf. 4.3). In this sensitivity analysis, we validated our results using a stricter filtering criterion, providing additional evidence to support the reliability of our findings. The analysis shows that our results remain consistent and robust even with stricter filtering criteria.

We also collect the ML baseline based on two keywords of "machine learning" and "deep learning". However, having these two keywords may not include all machine learning repositories on GitHub. Future work can expand our results by analyzing more projects extracted from more diverse keywords.

Internal validity We study the code quality metrics of the AIOps projects using a set of metrics (e.g., code smells). Nevertheless, these metrics may not accurately represent the quality of the projects. To reduce the effect of this threat, we leveraged a variety of metrics that represent different aspects of each project. These metrics have also been used in other articles to measure code quality (Businge et al. 2019; Lenarduzzi et al. 2019; Tan et al. 2018).

We rely on the code issues detected by SonarQube to answer our RQ3. We choose SonarQube since it is one of the most used tools for analyzing code quality (Lenarduzzi et al. 2017, 2018). We are aware that SonarQube might have false positives or false negatives. In order to evaluate the accuracy of code quality metrics extracted through SonarQube, we conduct an manual verification. We select the 10 most violated SonarQube rules in AIOps projects presented in Table 11. For each of the rules, we randomly select 20 issues from 20 different projects (from all the AIOps and baseline projects) where that rule violation is present. Therefore, for each of the 10 rules,20 instances are analyzed, and totally we analyze 200 issues. In our analysis, we look for false positives (i.e., any issue that SonarQube has detected but is not actually an issue). We did not find any false positives in the analyzed issues. Our results align with SonarQube's claim that it has zero false-positives for code smells and bugs.[8]

Construct validity To expand our set of AIOps projects, we perform pattern mining and choose 4 out of 194 pairs of keywords. We choose these 4 pairs of keywords after a systematic process and based on the discussions between the authors, in order to reduce the bias. All of the keywords are among the most frequently used keywords.

To answer RQ2, we use qualitative analysis to categorize the input data, analysis techniques, and goals of each AIOps project. Our results may be biased by personal deductions like any other qualitative study. In order to mitigate this threat, two authors of the paper performed the qualitative analysis carefully and followed a 5-step process to deduce the categories. We achieve a Cohen's kappa value of 0.84 which shows a strong and reliable agreement.

6 Related Work

This work studies AIOps projects on GitHub and analyzes the quality of the projects using SonarQube. Thus, we discuss the related work on the following three aspects.

6.1 AIOps Solutions

In recent years and with the emergence of AIOps, more and more studies have been conducted in this field. Prior works have come up with different solutions for various problems. Anomaly detection (e.g., Brown et al. 2018; Nedelkoski et al. 2019b) failure prediction (e.g., Notaro et al. 2021a; Zhao et al. 2021) ticket management (e.g., Xue et al. 2016; Xue et al. 2018 self-healing (e.g., Ding et al. 2014; Lou et al. 2013; Lou et al. 2017) and issue diagnosis (e.g., Lou et al. 2014) are among the most studied topics in AIOps solutions. We divide AIOps papers by their goals into five sections and discuss them in more detail.

Anomaly Detection Anomaly detection is one of the most common tasks in AIOps (Bogatinovski et al. 2021). Previous studies perform anomaly detection using different approaches; for example, Fu et al. (2009); Sharma et al. (2013) use clustering to find the anomalies, Liu et al. (2015); Xu et al. (2009) use tree-based models, and Brown et al. (2018); Du et al. (2017) use neural networks to perform the task. To perform anomaly detection, different data sources have been used. Even though event logs (Beschastnikh et al. 2014; Brown et al. 2018; Du et al. 2017; Fu et al. 2009) are the most common data source, other sources such as performance metrics (e.g., Sharma et al. 2013; Su et al. 2019 network traffic (e.g., Lakhina

[8] https://docs.sonarqube.org/latest/user-guide/rules/

et al. 2004; Lakhina et al. 2005) and traces (e.g., Nedelkoski et al. 2019a; Nedelkoski et al. 2019b) have also been employed.

Failure Prediction Predicting failures is also one of the most common tasks of AIOps (Notaro et al. 2021a). Failure prediction can be divided into two groups of hardware failures and system failures. Compared to anomaly detection tasks where various data sources have been employed, in failure prediction, usually only the performance metrics of the systems are used. Performance metrics are measurements that aid in identifying and analyzing system bottlenecks and diagnosing issues. Some of the most popular performance metrics include CPU utilization, memory utilization, response time, throughput, I/O, and network latency. Zhao et al. (2021) use different system metrics, such as CPU usage and the number of live threads of the system, to predict performance anomalies at run-time. Lin et al. (2018) leverage temporal data (e.g., CPU and memory utilization metrics, alerts) and spatial data (e.g., rack locations) and construct MING, a deep learning-based approach. MING is able to rank the faulty nodes of the cloud system. MING is applied to the maintenance of one of the cloud service systems in Microsoft. Li et al. (2020b) try to enhance the performance of MING, e.g., by enriching the data representing node failures through a novel oversampling approach. They also discuss some criteria for the successful adoption of AIOps solutions, including trust, interpretation, maintenance, scalability, and in-context evaluation.

Root Cause Analysis Root cause analysis is the approach of identifying the underlying causes of problems or issues in the system. The process involves identifying and analyzing the factors that contributed to the problem, determining the root cause, and developing solutions to address the underlying issue. Ding et al. (2021) leverage the generated alarms of a software system and find the root causes of the issues in an online manner. Wang et al (2021) implement a graph-based algorithm and construct GROOT to perform root cause analysis. Their approach uses a combination of different event data (i.e., performance metrics, logs, and developer activities). GROOT is deployed in the production services of eBay. Zhang et al. (2021) propose a root cause analysis framework that also leverages different data sources: Key Performance Indicators (KPIs), logs, and topology data. Their primary model is a hierarchical Bayesian network that helps handle novel types of root causes.

Incident management Incident management refers to the process of identifying, analyzing, and resolving incidents that occur within a software system. Although the majority of organizations have established unique methods for handling incidents, critical service incidents still happen unexpectedly, and the incident system fails to mitigate them. Chen et al. (2020b) provide an overview of incident management. They analyze the incident management practices at Microsoft over two years and identify the distribution of incident severities for each section (e.g., network, database). Saha and Hoi (2022) leverage past incident root cause analysis reports to manage the incidents. They present an incident causation analysis engine that extracts information from previous root cause analysis reports and use that knowledge for new incidents. They employ pre-trained NLP models over reports collected over a few years at Salesforce. Li et al. (2022b) propose an AIOps framework for incident detection. Their framework consists of four main parts: multi-aspect detection (where it is able to automatically identify the combinations of different data types such as logs and performance metrics), proactive detection (where it can proactively search for future hardware and software failures), incident refinement (where it can provide a global view of the incident and prioritize high-impact incidents), and incident enrichment (where it can locate the faulty scope).

AIOps literature reviews There exist two articles that survey existing AIOps studies. Notaro et al. (2021b) conduct a systematic mapping study to identify past research in AIOps. They provide a taxonomy of AIOps papers in order to investigate the trends and also a comparison of AIOps papers for specific problems. Their findings demonstrate an on-growing research interest in the field of AIOps, particularly for downstream tasks such as anomaly detection and root cause analysis. According to their paper, the majority of studies (more than 60%) are related to failure management (e.g., failure prediction and failure detection). Resource management and scheduling are two other popular areas in AIOps. To understand the trends in AIOps articles, similarly, Rijal et al. (2022) perform a literature review about AIOps works. Their findings support the growing interest in AIOps (Notaro et al. 2021b). Their results count several benefits of AIOps: better monitoring of IT work, efficient time saving, improved human-AI collaborations, proactive IT work, and faster Mean Time To Resolve (MTTR). The development of AIOps solutions also faces a number of difficulties, including doubt about the efficiency of AI and ML, low-quality data, identifying the proper use cases, and traditional engineering approaches. Their paper concludes the need for further research to improve human and AIOps interaction in order to enhance human productivity. The results of our work can complement these literature reviews.

However, none of the existing work studies real-world AIOps projects. Different from these studies, in this work, we perform an empirical study to understand the practices and characteristics of real-world AIOps projects on GitHub.

6.2 Characterizing GitHub Projects

As the biggest hosting service for open-source software, GitHub has been studied remarkably in the past years. Some studies focus on the best approaches for finding the most prominent repositories on GitHub (Dabic et al. 2021; Kalliamvakou et al. 2014, 2016). Many studies also have used GitHub as a source for mining software repositories (Businge et al. 2019; Coelho et al. 2018; Guzman et al. 2014; Horschig et al. 2018; Kallis et al. 2021; Lopes et al. 2017; Manes and Baysal 2021; Subramanian et al. 2020; Vadlamani and Baysal 2020; Wessel et al. 2018). For example, Vadlamani and Baysal (2020), along with Subramanian et al. (2020); Horschig et al. (2018) try to characterize the developers of open-source software. As another example, Kallis et al. (2021) try to predict the issue types of projects. More similar to our study, Ghrairi et al. (2018) study the state of Virtual Reality (VR) projects extracted from GitHub. In another work, Coppola et al. (2019) characterize the popularity of Kotlin in Android projects by analyzing 1,232 applications on GitHub. However, as far as we know, no studies have been conducted yet on the subject of AIOps. Our work is the first study that uses GitHub to characterize and analyze AIOps projects.

6.3 Analyzing Code Quality Using SonarQube

SonarQube is one of the most popular static code analyses used both in academia (Lenarduzzi et al. 2017, 2018) and industry (Vassallo et al. 2020). Previous research has conducted many analyses on the code quality of open-source software projects using SonarQube. Businge et al. (2019) use SonarQube and analyses the source code of 119 applications to find their number of bugs. In another work, Tan et al. (2018) study 9 Apache software systems written in Python to investigate their technical debt. Lenarduzzi et al. (2019) also investigate the

technical debt of 33 Apache systems written in Java. They find that the amount of code smell is much higher than bugs or vulnerabilities in their set of projects. They also report that the code smells with the severity level MAJOR take the longest time to get fixed. Compared to the mentioned studies, our paper analyzes the quality of 6,101 projects, which is much higher than in the previous studies, making our results more confident and robust.

7 Conclusion

This work studies the characteristics of AIOps projects on GitHub and conducts a comparative analysis with two baseline sets (ML and General projects). We combine both quantitative and qualitative analyses to understand the current state of AIOps solutions. Specifically, we illustrate the state of AIOps projects on GitHub in RQ1, observing that they are relatively new and have been growing rapidly in recent years. We determine the most common input data, techniques, and goals of AIOps solutions in RQ2. By uncovering these patterns, researchers and practitioners can learn from successful approaches and adopt optimal AIOps solutions for their specific application scenarios. We then investigate the quality of AIOps projects in terms of code quality and compare them to the baselines in RQ3. We observe that the quality of AIOps projects is poorer compared to the baselines. Our findings highlight the need for future efforts to enhance AIOps practices. For example, addressing weak aspects such as self-healing and reducing the *ad-hoc*ness of coding can lead to improved AIOps quality.

Our study could be extended in different ways. First, we study the AIOps projects publicly available on GitHub. However, there might be differences between open-source projects and closed-source projects which are implemented for private datasets in companies. Thus, one extension to our study could be to gather and analyze the industrial AIOps solutions and report any similarities and differences with AIOps projects on GitHub. Second, based on our results regarding the inputs, techniques, and goals of AIOps solutions, future work may develop a pipeline to automate or augment the development of AIOps solutions. Third, we discussed the potential of integrating existing quality assurance techniques from machine learning systems into AIOps solutions. Hence, future research could implement these techniques for AIOps solutions, examining their impact on enhancing AIOps code quality. Finally, conducting a future study that includes interviews with AIOps researchers and practitioners could serve to validate and extend the findings of our work.

Code Availability Our datasets, code and analysis are publicly available on https://github.com/roozbehaghili/studying_aiops_github.

Declarations

Conflicts of interest The authors have no conflicts of interest to declare that are relevant to the content of this article.

References

Artstein R, Poesio M (2008) Inter-coder agreement for computational linguistics. Comput Linguist 34(4):555–596

Basili VR, Selby RW, Hutchens DH (1986) Experimentation in software engineering. IEEE Trans Softw Eng 7:733–743

Beschastnikh I, Brun Y, Ernst MD, Krishnamurthy A (2014) Inferring models of concurrent systems from logs of their behavior with csight. In: Proceedings of the 36th International Conference on Software Engineering, p 468–479
Bogatinovski J, Nedelkoski S, Acker A, Schmidt F, Wittkopp T, Becker S, Cardoso J, Kao O (2021) Artificial intelligence for it operations (aiops) workshop white paper. arXiv:2101.06054
Braiek HB, Khomh F (2022) Testing feedforward neural networks training programs. ACM Trans Softw Eng Methodol. https://doi.org/10.1145/3529318
Brown A, Tuor A, Hutchinson B, Nichols N (2018) Recurrent neural network attention mechanisms for interpretable system log anomaly detection. In: Proceedings of the first workshop on machine learning for computing systems, p 1–8
Businge J, Openja M, Kavaler D, Bainomugisha E, Khomh F, Filkov V (2019) Studying android app popularity by cross-linking github and google play store. 2019 IEEE 26th International Conference on Software Analysis. Evolution and Reengineering (SANER), IEEE, pp 287–297
Chen B, Jiang ZM (2021) A survey of software log instrumentation. ACM Comput Surv (CSUR) 54(4):1–34
Chen Z, Cao Y, Liu Y, Wang H, Xie T, Liu X (2020a) A comprehensive study on challenges in deploying deep learning based software. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, p 750–762
Chen Z, Kang Y, Li L, Zhang X, Zhang H, Xu H, Zhou Y, Yang L, Sun J, Xu Z, et al. (2020b) Towards intelligent incident management: why we need it and how we make it. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, p 1487–1497
Chen Z, Liu J, Gu W, Su Y, Lyu MR (2021) Experience report: deep learning-based system log analysis for anomaly detection. arXiv:2107.05908
Cliff N (1993) Dominance statistics: Ordinal analyses to answer ordinal questions. Psychol Bull 114(3):494
Coelho J, Valente MT, Silva LL, Shihab E (2018) Identifying unmaintained projects in github. In: Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, pp 1–10
Cohen J (1960) A coefficient of agreement for nominal scales. Educ Psychol Meas 20(1):37–46
Coppola R, Ardito L, Torchiano M (2019) Characterizing the transition to kotlin of android apps: a study on f-droid, play store, and github. In: Proceedings of the 3rd ACM SIGSOFT International Workshop on App Market Analytics, p 8–14
Dabic O, Aghajani E, Bavota G (2021) Sampling projects in github for msr studies. In: 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR), IEEE, pp 560–564
Dakhel AM, Desmarais MC, Khomh F (2023) Dev2vec: Representing domain expertise of developers in an embedding space. Inf Softw Technol 159:107218
Dang Y, Lin Q, Huang P (2019) Aiops: real-world challenges and research innovations. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), IEEE, p 4–5
Devlin J, Chang MW, Lee K, Toutanova K (2018) Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805
Di Stefano A, Di Stefano A, Morana G, Zito D (2021) Prometheus and aiops for the orchestration of cloud-native applications in ananke. In: 2021 IEEE 30th International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE), IEEE, p 27–32
Ding J, Wang X, Ye X, Ouyang Y, Chai Y (2021) Ttercl: An onsite real-time alarm root-cause location algorithm. In: 2021 IEEE International Conference on Big Data (Big Data), IEEE, p 4852–4858
Ding R, Fu Q, Lou JG, Lin Q, Zhang D, Shen J, Xie T (2012) Healing online service systems via mining historical issue repositories. In: Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering, ASE -12, p 318–321
Ding R, Fu Q, Lou JG, Lin Q, Zhang D, Xie T (2014) Mining historical issue repositories to heal large-scale online service systems. In: 2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, IEEE, p 311–322
Du M, Li F, Zheng G, Srikumar V (2017) Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security, p 1285–1298
El-Sayed N, Zhu H, Schroeder B (2017) Learning from failure across multiple clusters: A trace-driven approach to understanding, predicting, and mitigating job terminations. In: 2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS), IEEE, p 1333–1344
Feng Z, Guo D, Tang D, Duan N, Feng X, Gong M, Shou L, Qin B, Liu T, Jiang D, et al. (2020) Codebert: A pre-trained model for programming and natural languages. arXiv:2002.08155

Foalem PL, Khomh F, Li H (2023) Studying logging practice in machine learning-based applications. arXiv:2301.04234
Fu Q, Lou JG, Wang Y, Li J (2009) Execution anomaly detection in distributed systems through unstructured log analysis. In: 2009 ninth IEEE international conference on data mining, IEEE, p 149–158
Gao J, Wang H, Shen H (2020) Task failure prediction in cloud data centers using deep learning. IEEE Trans Serv Comput
Ghrairi N, Kpodjedo S, Barrak A, Petrillo F, Khomh F (2018) The state of practice on virtual reality (vr) applications: An exploratory study on github and stack overflow. 2018 IEEE International Conference on Software Quality. Reliability and Security (QRS), IEEE, pp 356–366
GitHub (2023) GitHub. https://github.com/, Last Accessed: 06-June-2023
GitHub I (2022) To create integrations, retrieve data, and automate your workflows, build with the github rest api. https://docs.github.com/en/rest
Guzman E, Azócar D, Li Y (2014) Sentiment analysis of commit comments in github: an empirical study. In: Proceedings of the 11th working conference on mining software repositories, p 352–355
Han J, Pei J, Yin Y (2000) Mining frequent patterns without candidate generation. ACM sigmod record 29(2):1–12
Han J, Pei J, Yin Y, Mao R (2004) Mining frequent patterns without candidate generation: A frequent-pattern tree approach. Data Min Knowl Discov 8(1):53–87
Han J, Cheng H, Xin D, Yan X (2007) Frequent pattern mining: current status and future directions. Data Min Knowl Discov 15(1):55–86
He P, Chen Z, He S, Lyu MR (2018a) Characterizing the natural language descriptions in software logging statements. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, p 178–189
He S, Zhu J, He P, Lyu MR (2016) Experience report: System log analysis for anomaly detection. In: 2016 IEEE 27th international symposium on software reliability engineering (ISSRE), IEEE, p 207–218
He S, Lin Q, Lou JG, Zhang H, Lyu MR, Zhang D (2018b) Identifying impactful service system problems via log analysis. In: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, p 60-70
Hindle A, Barr ET, Gabel M, Su Z, Devanbu P (2016) On the naturalness of software. Commun ACM 59(5):122–131
Horschig S, Mattis T, Hirschfeld R (2018) Do java programmers write better python? studying off-language code quality on github. In: Conference companion of the 2nd international conference on art, science, and engineering of programming, p 127–134
Islam MJ, Nguyen G, Pan R, Rajan H (2019) A comprehensive study on deep learning bug characteristics. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, p 510–520
Jupyter I (2022) nbconvert: Convert Notebooks to other formats. https://nbconvert.readthedocs.io/en/latest/, Last Accessed: 06-Dec.-2022
Kalliamvakou E, Gousios G, Blincoe K, Singer L, German DM, Damian D (2014) The promises and perils of mining github. In: Proceedings of the 11th working conference on mining software repositories, p 92–101
Kalliamvakou E, Gousios G, Blincoe K, Singer L, German DM, Damian D (2016) An in-depth study of the promises and perils of mining github. Empir Softw Eng 21(5):2035–2071
Kallis R, Di Sorbo A, Canfora G, Panichella S (2021) Predicting issue types on github. Sci Comput Program 205:102598
Khandkar SH (2009) Open coding. University of Calgary 23:2009
Kodinariya TM, Makwana PR et al (2013) Review on determining number of cluster in k-means clustering. Int J 1(6):90–95
Lakhina A, Crovella M, Diot C (2004) Diagnosing network-wide traffic anomalies. ACM SIGCOMM Comput Commun Rev 34(4):219–230
Lakhina A, Crovella M, Diot C (2005) Mining anomalies using traffic feature distributions. ACM SIGCOMM Comput Commun Rev 35(4):217–228
Lenarduzzi V, Sillitti A, Taibi D (2017) Analyzing forty years of software maintenance models. In: 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C), IEEE, p 146–148
Lenarduzzi V, Sillitti A, Taibi D (2018) A survey on code analysis tools for software maintenance prediction. In: International Conference in Software Engineering for Defence Applications, Springer, p 165–175
Lenarduzzi V, Saarimaki N, Taibi D (2019) On the diffuseness of code technical debt in java projects of the apache ecosystem. In: 2019 IEEE/ACM International Conference on Technical Debt (TechDebt), IEEE, p 98–107

Li J, Dai L, Tan F, Shen H, Wang Z, Sheng B, Hu P (2022) Cdx-net: Crossdomain multi-feature fusion modeling via deep neural networks for multivariate time series forecasting in aiops. ICASSP 2022–2022 IEEE International Conference on Acoustics. Speech and Signal Processing (ICASSP), IEEE, pp 4073–4077

Li S, Wu Y, Liu Y, Wang D, Wen M, Tao Y, Sui Y, Liu Y (2020a) An exploratory study of bugs in extended reality applications on the web. In: 2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE), IEEE, p 172–183

Li Y, Jiang ZM, Li H, Hassan AE, He C, Huang R, Zeng Z, Wang M, Chen P (2020) Predicting node failures in an ultra-large-scale cloud computing platform: an aiops solution. ACM Trans Softw Eng Methodol (TOSEM) 29(2):1–24

Li Y, Zhang X, He S, Chen Z, Kang Y, Liu J, Li L, Dang Y, Gao F, Xu Z et al (2022) An intelligent framework for timely, accurate, and comprehensive cloud incident detection. ACM SIGOPS Oper Syst Rev 56(1):1–7

Lim MH, Lou JG, Zhang H, Fu Q, Teoh ABJ, Lin Q, Ding R, Zhang D (2014) Identifying recurrent and unknown performance issues. In: 2014 IEEE International Conference on Data Mining, IEEE, p 320–329

Lin Q, Hsieh K, Dang Y, Zhang H, Sui K, Xu Y, Lou JG, Li C, Wu Y, Yao R, et al. (2018) Predicting node failure in cloud service systems. In: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, p 480–490

Liu D, Zhao Y, Xu H, Sun Y, Pei D, Luo J, Jing X, Feng M (2015) Opprentice: Towards practical and automatic anomaly detection through machine learning. In: Proceedings of the 2015 internet measurement conference, p 211–224

Lomio F, Moreschini S, Lenarduzzi V (2021) Fault prediction based on software metrics and SonarQube rules. machine or deep learning? arXiv:2103.11321

Lopes CV, Maj P, Martins P, Saini V, Yang D, Zitny J, Sajnani H, Vitek J (2017) Déjàvu: a map of code duplicates on github. Proceedings of the ACM on Programming Languages 1(OOPSLA):1–28

Lou JG, Lin Q, Ding R, Fu Q, Zhang D, Xie T (2013) Software analytics for incident management of online services: An experience report. In: 2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, p 475–485

Lou JG, Lin Q, Ding R, Fu Q, Zhang D, Xie T (2017) Experience report on applying software analytics in incident management of online service. Autom Softw Eng 24(4):905–941

Lovrić M, Milanović M, Stamenković M (2014) Algoritmic methods for segmentation of time series: An overview. J Contemp Econ Bus Issues 1(1):31–53

Luo C, Lou JG, Lin Q, Fu Q, Ding R, Zhang D, Wang Z (2014) Correlating events with time series for incident diagnosis. In: Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining, p 1583–1592

Lyu Y, Rajbahadur GK, Lin D, Chen B, Jiang ZM (2021) Towards a consistent interpretation of aiops models. ACM Trans Softw Eng Methodol (TOSEM) 31(1):1–38

Majidi F, Openja M, Khomh F, Li H (2022) An empirical study on the usage of automated machine learning tools. In: 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, p 59–70

Manes SS, Baysal O (2021) Studying the change histories of stack overflow and github snippets. In: 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR), IEEE, p 283–294

Mann HB, Whitney DR (1947) On a test of whether one of two random variables is stochastically larger than the other. The annals of mathematical statistics p 50–60

McHugh ML (2012) Interrater reliability: the kappa statistic. Biochemia medica 22(3):276–282

Munaiah N, Kroh S, Cabrey C, Nagappan M (2017) Curating github for engineered software projects. Empir Softw Eng 22(6):3219–3253

Nakajima S (2018) Quality assurance of machine learning software. In: 2018 IEEE 7th Global Conference on Consumer Electronics (GCCE), IEEE, p 601–604

Nedelkoski S, Cardoso J, Kao O (2019a) Anomaly detection and classification using distributed tracing and deep learning. In: 2019 19th IEEE/ACM international symposium on cluster, cloud and grid computing (CCGRID), IEEE, p 241–250

Nedelkoski S, Cardoso J, Kao O (2019b) Anomaly detection from system tracing data using multimodal deep learning. In: 2019 IEEE 12th International Conference on Cloud Computing (CLOUD), IEEE, p 179–186

Nikanjam A, Braiek HB, Morovati MM, Khomh F (2021) Automatic fault detection for deep learning programs using graph transformations. ACM Trans Softw Eng Methodol 31(1), https://doi.org/10.1145/3470006

Nikanjam A, Morovati MM, Khomh F, Ben Braiek H (2022) Faults in deep reinforcement learning programs: A taxonomy and a detection approach. Autom Softw Eng 29(1), https://doi.org/10.1007/s10515-021-00313-x

Notaro P, Cardoso J, Gerndt M (2021) A survey of aiops methods for failure management. ACM Trans Intell Syst Technol (TIST) 12(6):1–45

Notaro P, Cardoso J, Gerndt M (2021b) A systematic mapping study in aiops. In: Service-Oriented Computing-ICSOC 2020 Workshops: AIOps, CFTIC, STRAPS, AI-PA, AI-IOTS, and Satellite Events, Dubai, United Arab Emirates, December 14-17, 2020, Proceedings, Springer, p 110–123

Openja M, Morovati MM, An L, Khomh F, Abidi M (2022) Technical debts and faults in open-source quantum software systems: An empirical study. J Syst Softw 193:111458

Poth A, Meyer B, Schlicht P, Riel A (2020) Quality assurance for machine learning-an approach to function and system safeguarding. 2020 IEEE 20th International Conference on Software Quality. Reliability and Security (QRS), IEEE, pp 22–29

Prasad P, Rich C (2018) Market Guide for AIOps Platforms. https://www.gartner.com/en/documents/3892967, Last Accessed: -06-Dec.-2022

Rasool G, Arshad Z (2015) A review of code smell mining techniques. J Softw: Evol Process 27(11):867–895

Rijal L, Colomo-Palacios R, Sánchez-Gordón M (2022) Aiops: A multivocal literature review. Artificial Intelligence for Cloud and Edge Computing p 31–50

Romano J, Kromrey JD, Coraggio J, Skowronek J, Devine L (2006) Exploring methods for evaluating group differences on the nsse and other surveys: Are the t-test and cohensd indices the most appropriate choices. In: annual meeting of the Southern Association for Institutional Research, Citeseer, p 1–51

Rosá A, Chen LY, Binder W (2015) Catching failures of failures at big-data clusters: A two-level neural network approach. In: 2015 IEEE 23rd International Symposium on Quality of Service (IWQoS), IEEE, p 231–236

Saarimaki N, Baldassarre MT, Lenarduzzi V, Romano S (2019) On the accuracy of SonarQube technical debt remediation time. In: 2019 45th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), IEEE, p 317–324

Saha A, Hoi SC (2022) Mining root cause knowledge from cloud service incident investigations for aiops. In: Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice, p 197–206

Shapiro SS, Wilk MB (1965) An analysis of variance test for normality (complete samples). Biometrika 52(3/4):591–611

Sharma B, Jayachandran P, Verma A, Das CR (2013) Cloudpd: Problem determination and diagnosis in shared dynamic clouds. In: 2013 43rd Annual EEE/IFIP International Conference on Dependable Systems and Networks (DSN), IEEE, p 1–12

Stol KJ, Ralph P, Fitzgerald B (2016) Grounded theory in software engineering research: a critical review and guidelines. In: Proceedings of the 38th International conference on software engineering, p 120–131

Su Y, Zhao Y, Niu C, Liu R, Sun W, Pei D (2019) Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In: Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining, p 2828–2837

Subramanian VN, Rehman I, Nagappan M, Kula RG (2020) Analyzing first contributions on github: what do newcomers do. IEEE Software

Svoboda J, Ghafir I, Prenosil V et al (2015) Network monitoring approaches: An overview. Int J Adv Comput Netw Secur 5(2):88–93

Tambon F, Khomh F, Antoniol G (2023) A probabilistic framework for mutation testing in deep neural networks. Inf Softw Technol 155:107129. https://doi.org/10.1016/j.infsof.2022.107129, https://www.sciencedirect.com/science/article/pii/S0950584922002385

Tan J, Lungu M, Avgeriou P (2018) Towards studying the evolution of technical debt in the python projects from the apache software ecosystem. In: BENEVOL, p 43–45

Tu Z, Su Z, Devanbu P (2014) On the localness of software. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, p 269–280

Vadlamani SL, Baysal O (2020) Studying software developer expertise and contributions in stack overflow and github. In: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, p 312–323

Vassallo C, Panichella S, Palomba F, Proksch S, Gall HC, Zaidman A (2020) How developers engage with static analysis tools in different contexts. Empir Softw Eng 25(2):1419–1457

Wang H, Wu Z, Jiang H, Huang Y, Wang J, Kopru S, Xie T (2021) Groot: An event-graph-based approach for root cause analysis in industrial settings. In: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, p 419–429

Wessel M, De Souza BM, Steinmacher I, Wiese IS, Polato I, Chaves AP, Gerosa MA (2018) The power of bots: Characterizing and understanding bots in oss projects. Proceedings of the ACM on Human-Computer Interaction 2(CSCW):1–19

Wilson SJ (2017) Data representation for time series data mining: time domain approaches. Wiley Interdiscip Rev Comput Stat 9(1):e1392

Wohlin C, Aurum A (2015) Towards a decision-making structure for selecting a research design in empirical software engineering. Empir Softw Eng 20(6):1427–1455

Xu W, Huang L, Fox A, Patterson D, Jordan MI (2009) Detecting largescale system problems by mining console logs. In: Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles, p 117–132

Xue J, Birke R, Chen LY, Smirni E (2016) Managing data center tickets: Prediction and active sizing. In: 2016 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), IEEE, p 335–346

Xue J, Birke R, Chen LY, Smirni E (2018) Spatial-temporal prediction models for active ticket managing in data centers. IEEE Trans Netw Serv Manag 15(1):39–52

Yao K, Li H, Shang W, Hassan AE (2020) A study of the performance of general compressors on log files. Empir Softw Eng 25:3043–3085

Zhang T, Gao C, Ma L, Lyu M, Kim M (2019) An empirical study of common challenges in developing deep learning applications. In: 2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE), IEEE, p 104–115

Zhang Y, Chen Y, Cheung SC, Xiong Y, Zhang L (2018) An empirical study on tensorflow program bugs. In: Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, p 129–140

Zhang Y, Guan Z, Qian H, Xu L, Liu H, Wen Q, Sun L, Jiang J, Fan L, Ke M (2021) Cloudrca: a root cause analysis framework for cloud computing platforms. In: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, p 4373–4382

Zhao G, Hassan S, Zou Y, Truong D, Corbin T (2021) Predicting performance anomalies in software systems at run-time. ACM Trans Softw Eng Methodol (TOSEM) 30(3):1–33

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.